MORSE-BASED MODULAR HOMOLOGY FOR EVOLVING SIMPLICIAL COMPLEXES

A PREPRINT

Anqiao Ouyang
Mount Boucherie Secondary School
West Kelowna, British Columbia, Canada

Department of Aerospace Engineering
San Diego State University
San Diego, California, USA
aouyang@sdsu.edu

October 12, 2026

ABSTRACT

We study the maintenance of the Betti numbers over $\mathbb{Z}_2$ of a simplicial complex under insertions and deletions of simplices. For each dimension we maintain a reduced matrix $R = BV$ together with the change of basis V, a dynamic reduced basis. An inserted simplex is reduced against the existing pivots; a deleted simplex is removed from every basis chain that contains it, by adding either a cycle or the column with the smallest pivot, the rank-only form of known updates for zigzag persistence and warm-started reductions. We prove that the Betti numbers are exact after every edit, for every order of the cells. In the order given by a collapse, every gradient pair is an apparent pair, and a build performs column additions only in the columns of critical cells, where they coincide with the reduction of the Morse complex. The cost of an edit is output-sensitive, and we exhibit families on which every toggle of a single simplex costs $\Omega(n)$, so no bound in terms of the size of an edit holds. On five workload families with up to 3,932,162 simplices, every Betti vector reported by an exact method was checked against a reference computation and none disagreed. Where edits create and destroy only small cycles, in refined and carved 3-balls and in flag complexes, the latency hardly depends on the size of the complex: at 955,059 simplices every variant is 90.2–91.5 times faster than static recomputation, with or without the Morse order. On punctured surfaces the method is at most 2.06 times faster on average and has the worse tail latency, and combinatorial counters are far cheaper; on the non-locality witnesses its cost grows linearly with the complex. The Morse order saves nothing at build time against structured insertion orders but much against a random one, and helps between rebuilds only on some workloads.

Keywords Simplicial Homology, Betti Numbers, Dynamic Homology Computation, Discrete Morse Theory, Boundary Matrix Reduction, Computational Algebraic Topology

1 Introduction

The Betti numbers $\beta_0, \beta_1, \beta_2, \beta_3$ of a simplicial complex count its connected components, independent loops, enclosed voids and, in dimension three, independent 3-cycles; over a field they are determined by the ranks of the boundary matrices [17, 24, 35]. In many uses the complex is not fixed. Adaptive meshes are refined and coarsened by local moves [37], cavities are carved into a volume mesh and refilled, holes are opened in a surface and closed again, and the proximity complex of a point cloud changes as the points move [6]. After each modification one wants to know whether it changed the topology. Recomputing the Betti numbers from scratch answers this, but it reads the whole complex after every modification, however small the modification is.

Over a field, $\beta_k = n_k - r_k - r_{k+1}$, where n_k is the number of k-simplices and r_k the rank of the k-th boundary matrix B_k. Maintaining Betti numbers therefore means maintaining ranks, and the problem has more structure than general dynamic rank: the boundary of a simplex does not change while the simplex exists, so each atomic edit inserts or deletes one column of one boundary matrix, and rows appear and disappear only while they are zero (Observation 2.1). Insertions alone are handled by reducing the new column against the existing pivots: each inserted simplex either creates a cycle or destroys one [11]. Deletions are the difficulty. After reduction, a column is a sum of original columns; when one of these is deleted, every reduced column that absorbed it becomes invalid, and the reduced matrix alone does not record which columns these are. This paper maintains that record and studies what it costs.

Contributions.

(i) *Exact dynamic reduced basis maintenance.* For each dimension we maintain a reduced matrix $R = B_k V$ together with the change of basis V and, for every simplex, the columns of V that contain it. Insertion is the standard incremental reduction; a deletion corrects every basis chain that contains the deleted simplex, by a cycle (Case A) or by the column with the smallest pivot (Case B). We prove that the Betti numbers over $\mathbb{Z}_2$ are exact after every edit, for every order of the cells and in every dimension, and that initialisation and rebuilds by the static reduction with clearing produce a valid state (Theorem 1 and Proposition 1.1). The deletion step is the rank-only specialisation of the deletion updates used for zigzag persistence and warm-started $R = DV$ factorisations [8, 22, 28]; we give a self-contained proof.

(ii) *A Morse-induced order and its properties.* Ordering the cells by reverse removal time in a collapse makes every gradient pair an apparent pair (Lemma 2, close to known results), so a build performs column additions only in the columns of critical cells (Corollary 2.1), and on those columns it performs exactly the reduction of the Morse complex (Corollary 2.2). Between rebuilds the order is a heuristic, and we measure its effect.

(iii) *Output-sensitive costs and the limits of locality.* The cost of an edit is stated in terms of footprints and reduction chains (Proposition 3). On two explicit families, a cycle graph and a triangulated sphere, every toggle of one simplex costs this data structure $\Omega(n)$ operations (Proposition 4), so no bound in terms of the size of an edit holds.

(iv) *An evaluation that identifies where the method is effective.* On five workload families (punctured surfaces; refinement, coarsening and cavities in a 3-ball; flag complexes of moving points; non-locality witnesses) with up to 3,932,162 simplices, we compare the method with static recomputation in insertion order, in a canonical order that cannot drift with the history, and in GUDHI [30], and with ablations of the order and of the rebuild policy (Section 6). Every Betti vector reported by an exact method was checked against at least one reference computation, 98.3 % against one outside our engine; none disagreed. Where edits create and destroy only small cycles, the update latency hardly depends on the size of the complex: on the largest refinement instance (955,059 simplices) every variant of the dynamic reduced basis is 90.2–91.5 times faster than static recomputation, with or without the Morse order. On punctured surfaces it is at most 2.06 times faster on average, some variants are slower, every variant has a worse tail latency, and combinatorial counters are far cheaper; on the non-locality witnesses its cost grows linearly with the complex. At build time the Morse order saves nothing against the structured insertion orders of our generators, although any structured order is far better than a random one; rebuilding pays off where the basis grows, as on cavities, and not everywhere.

Scope. All statements are over $\mathbb{Z}_2$; we say nothing about integer homology or torsion. The maintained basis yields ranks and a basis of each cycle space, but no persistence pairing. We prove no bound on the cost of an update in terms of its size, and we make no sparsity claim for the state between rebuilds.

Outline. Section 2 states the problem and gives an overview of the algorithm. Section 3 defines the dynamic reduced basis and proves its exactness, Section 4 introduces the Morse order and its properties, and Section 5 analyses the cost of an update and proves that no locality bound holds. Section 6 reports the experiments, Section 7 discusses limitations and when to use which method, Section 8 reviews related work, and Section 9 concludes.

2 Problem and algorithm overview

2.1 Setting

Let K be a finite simplicial complex of dimension d. A k-simplex is a set of $k+1$ vertices; K_k is the set of k-simplices of K and $n_k = |K_k|$. We identify the chain group $C_k = C_k(K; \mathbb{Z}_2)$ with the set of finite subsets of K_k, addition being symmetric difference. The boundary $\partial_k \colon C_k \to C_{k-1}$ sends a simplex to the set of its facets (codimension-one faces); B_k is its matrix, with rows indexed by K_{k-1} and columns by K_k. We write $Z_k = \ker \partial_k$, $r_k = \operatorname{rank} B_k$ for $1 \le k \le d$,

and $r_0 = r_{d+1} = 0$. By rank–nullity, $\dim Z_k = n_k - r_k$ and $\dim \operatorname{im} \partial_{k+1} = r_{k+1}$, so

$$\beta_k \;=\; n_k - r_k - r_{k+1} \qquad (0 \le k \le d). \tag{1}$$

Edits and updates. An *atomic edit* is either INSERT(σ), for a simplex $\sigma \notin K$ all of whose facets lie in K, or DELETE(σ), for a simplex $\sigma \in K$ that has no proper coface in K. Each atomic edit leaves a simplicial complex. An *update* u_t is a finite sequence of atomic edits (a batch); its *size* is $s_t = |u_t|$. The task is to output $\beta_0, \dots, \beta_d$ of K_t after every update t.

Observation 2.1 (Structural fact)**.** *Let σ be a k-simplex. While $\sigma \in K$, its boundary $\partial\sigma$ does not change. Inserting σ appends the column $\partial\sigma$ to B_k and appends a zero row (indexed by σ) to B_{k+1}; deleting σ removes its column from B_k and removes its row from B_{k+1}, and that row is zero. No other boundary matrix changes, and among $n_0, \dots, n_d$ only n_k changes.*

Proof. The boundary of a simplex is determined by its vertex set. When σ is inserted it has no coface: a coface would already be in K, and K is closed under faces while $\sigma \notin K$. A deleted simplex has no coface by the precondition of DELETE. In both cases the row σ of B_{k+1} is zero. □

Consequently, for each $k \ge 1$ the problem is to maintain $r_k = \operatorname{rank} B_k$ under *column insertion and column deletion*, where rows appear and disappear only while they are zero; adding or removing a zero row changes no rank. The vertex dimension $k = 0$ needs no matrix: $\partial_0 = 0$.

2.2 The algorithm at a glance

For each dimension $k \ge 1$ we store a reduced matrix $R = B_k V$ together with the invertible change of basis V, and for every k-simplex σ the set $\operatorname{occ}(\sigma)$ of columns of V that contain it, the *footprint* of σ (Figure 1). An insertion reduces the new column against the existing pivots, as in the standard persistence algorithm [18, 39]. A deletion rewrites every column in the footprint of the deleted simplex: if one of them is a cycle, that cycle is added to the others (Case A); otherwise the column with the smallest pivot is added to the others (Case B). Section 3 proves that this keeps the Betti numbers exact after every atomic edit, for every order of the cells. The order affects only the cost. When the structure is built or rebuilt we compute an acyclic matching by collapses and order the cells by reverse removal time (Section 4); under this order the static reduction performs all its column additions in the columns of critical cells. Between rebuilds the order is a heuristic for a sparse basis. Section 5 states the cost of an edit in output-sensitive terms and shows that it admits no bound in terms of the size of the edit. Coefficients are in $\mathbb{Z}_2$ throughout. The proofs hold in every dimension; the implementation and the experiments are restricted to $\dim K \le 3$.

3 The dynamic reduced basis

Every cell $x \in K$ carries an *order key* $\operatorname{ord}(x)$; keys of distinct cells are distinct. For a non-zero chain c, $\operatorname{low}(c)$ is the cell of c with the largest key. Keys do not change between rebuilds (Section 4); an inserted cell receives a key larger than every key in use. Nothing in this section depends on how the keys are chosen; Section 4 chooses them.

Definition 3.1 (DRB)**.** Let $k \ge 1$. A *dynamic reduced basis* of B_k is a finite family of *slots* $j \in J_k$, each holding a pair (R_j, V_j) with $V_j \in C_k$ and $R_j \in C_{k-1}$, such that

(I1) $R_j = \partial_k V_j$ for every $j \in J_k$;
(I2) the non-zero R_j have pairwise distinct lows;
(I3) $(V_j)_{j \in J_k}$ is a basis of C_k; in particular $|J_k| = n_k$.

Two index structures are derived from the slots: $\operatorname{pivot}(\ell) = j$ if $R_j \neq 0$ and $\operatorname{low}(R_j) = \ell$ (well defined by (I2)), and, for $x \in K_k$, $\operatorname{occ}(x) = \{j \in J_k : x \in V_j\}$, the *footprint* of x. Both are updated together with the chains; we do not spell out this bookkeeping.

Writing R and V for the matrices with columns R_j and V_j, (I1) and (I3) say that $R = B_k V$ with V invertible, and (I2) says that R is *reduced*. Unlike the decomposition used for persistence, V is not required to be triangular with respect to ord.

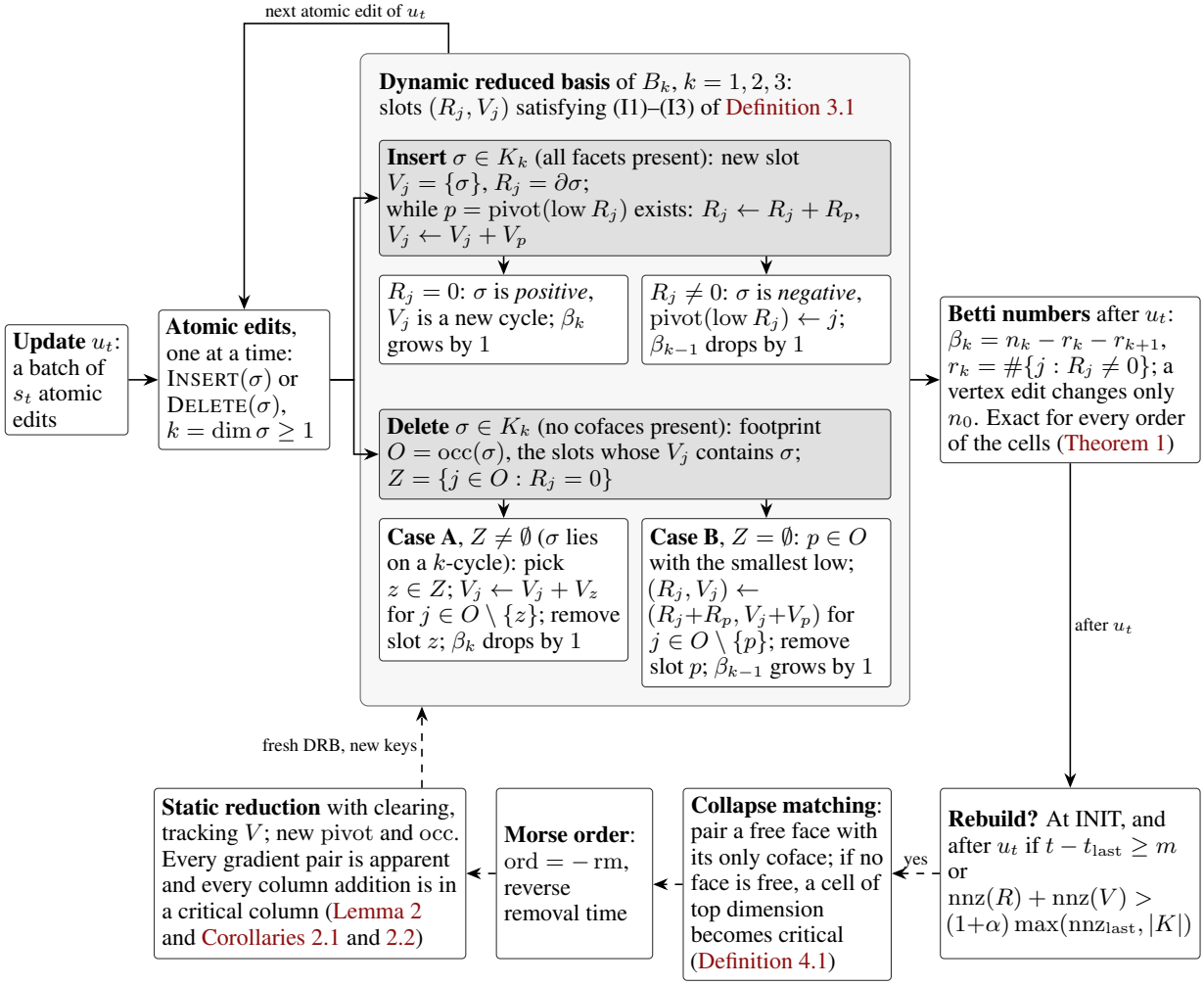


Figure 1: The method. *Top:* an update is processed one atomic edit at a time on the dynamic reduced basis of each boundary matrix (Section 3). An insertion reduces the new column against the pivots; a deletion rewrites every slot in the footprint $\text{occ}(\sigma)$ of the deleted simplex, by a cycle (Case A) or by the slot with the smallest low (Case B). The Betti numbers are exact after every edit, whatever the order of the cells. *Bottom, dashed:* a rebuild replaces the order by the Morse order of a collapse matching and recomputes the basis by the static reduction with clearing (Section 4). Under that order every gradient pair is an apparent pair and all column additions happen in columns of critical cells. Between rebuilds the Morse order is only a heuristic for sparsity (Remark 4.3). The ablations of Section 6.2 change the bottom path: drb-id-rb rebuilds in insertion order, and drb-id and mmhm-norb never rebuild.

Insertion. INSERT(σ) for a k-simplex $\sigma \notin K$, $k \geq 1$, all of whose facets are in K:

1. give σ a key larger than all keys in use; create a slot j with $V_j = \{\sigma\}$ and $R_j = \partial\sigma$;
2. while $R_j \neq 0$ and $p = \text{pivot}(\text{low}(R_j))$ is defined: $R_j \leftarrow R_j + R_p$, $V_j \leftarrow V_j + V_p$;
3. if $R_j \neq 0$, set $\text{pivot}(\text{low}(R_j)) = j$.

The slots $p_1, \ldots, p_m$ added in step 2 form the *reduction chain* of σ. We call σ *positive* if $R_j = 0$ at the end and *negative* otherwise.

Deletion. DELETE(σ) for $\sigma \in K_k$, $k \geq 1$, without cofaces. Let $O = \text{occ}(\sigma)$ and $Z = \{j \in O : R_j = 0\}$.

Case A ($Z \neq \emptyset$). Choose any $z \in Z$ (the implementations take one with $|V_z|$ minimal, ties broken by the smallest slot index). For every $j \in O \setminus \{z\}$: $V_j \leftarrow V_j + V_z$. Remove slot z.

Case B ($Z = \emptyset$). Let $p \in O$ be the slot whose $\text{low}(R_p)$ has the smallest key. For every $j \in O \setminus \{p\}$: $R_j \leftarrow R_j + R_p$ and $V_j \leftarrow V_j + V_p$. Remove slot p and the entry $\text{pivot}(\text{low}(R_p))$.

Finally remove σ from K. Inserting or deleting a vertex changes only n_0.

Theorem 1 (Exactness). *Let $k \geq 1$ and let $(R_j, V_j)_{j \in J_k}$ satisfy (I1)–(I3) for $B_k(K)$, for some injective key function.*

(a) $r_k = \#\{j \in J_k : R_j \neq 0\}$, and $\{V_j : R_j = 0\}$ is a basis of $Z_k(K)$.

(b) Let $\sigma \notin K$ be a k-simplex whose facets are all in K. The loop of INSERT(σ) *terminates, and afterwards the slots satisfy (I1)–(I3) for $B_k(K \cup \{\sigma\})$; every family satisfying (I1)–(I3) for $B_{k+1}(K)$ satisfies them, unchanged, for $B_{k+1}(K \cup \{\sigma\})$.*

(c) Let $\sigma \in K_k$ have no coface. Then $O \neq \emptyset$; after DELETE(σ)*, in Case A and in Case B, the slots satisfy (I1)–(I3) for $B_k(K \setminus \{\sigma\})$; every family satisfying (I1)–(I3) for $B_{k+1}(K)$ satisfies them, unchanged, for $B_{k+1}(K \setminus \{\sigma\})$.*

(d) Inserting a positive k-simplex increases β_k by one; inserting a negative one decreases β_{k-1} by one. A deletion in Case A decreases β_k by one; a deletion in Case B increases β_{k-1} by one. No other Betti number changes. Moreover, DELETE(σ) *is in Case A if and only if σ lies in the support of a k-cycle of K, and* INSERT(σ) *is positive if and only if σ lies in the support of a k-cycle of $K \cup \{\sigma\}$.*

Proof. (a) By (I1) and (I3), $R = B_k V$ with V invertible, so $\operatorname{rank} R = r_k$. The non-zero columns of R are linearly independent: in a non-empty sum of them, let R_i be the summand whose low ℓ has the largest key; every other summand has its low, hence all its cells, strictly below ℓ, so ℓ survives in the sum. Hence r_k is the number of non-zero R_j. Each V_j with $R_j = 0$ is a cycle; these chains are independent as part of a basis, and there are $n_k - r_k = \dim Z_k$ of them.

(b) Let $K' = K \cup \{\sigma\}$. Throughout the loop, $R_j = \partial V_j$ (true at the start and preserved by linearity because $R_p = \partial V_p$) and $V_j = \sigma + w$ with $w \in C_k(K)$. An addition is performed only when $\operatorname{low}(R_p) = \operatorname{low}(R_j) = \ell$; all cells of both chains have keys at most $\operatorname{ord}(\ell)$ and ℓ cancels, so afterwards $R_j = 0$ or $\operatorname{low}(R_j)$ is strictly smaller. Hence the loop terminates, after at most one addition per non-zero slot. On exit either $R_j = 0$ or $\operatorname{low}(R_j)$ is no pivot, so recording the pivot preserves (I2); (I1) holds. Since $C_k(K') = C_k(K) \oplus \mathbb{Z}_2\sigma$ and the old V_i form a basis of $C_k(K)$, the old chains together with $V_j = \sigma + w$ form a basis of $C_k(K')$: they are $n_k + 1$ chains spanning $\sigma = V_j + w$. For B_{k+1}: the chain group C_{k+1} is unchanged, each $R_i = \partial V_i$ is supported on facets of $(k+1)$-simplices of K', none of which is σ (Observation 2.1), and keys of existing cells are unchanged; so (I1)–(I3) continue to hold.

(c) Since the V_j span C_k, σ is a sum of some of them, so some V_j contains σ: $O \neq \emptyset$. Each operation $V_j \leftarrow V_j + V_q$ with $j \neq q$ is an invertible change of basis, so throughout the loop the family remains a basis of $C_k(K)$. For $j \in O \setminus \{q\}$ (with $q = z$ in Case A, $q = p$ in Case B), both V_j and V_q contain σ, so $V_j + V_q$ does not; slots outside O never contained σ. After the loop, V_q is therefore the only chain containing σ. Removing slot q leaves $n_k - 1$ independent chains, all in $C_k(K \setminus \{\sigma\})$, which has dimension $n_k - 1$: (I3) holds. *Case A.* Because $\partial V_z = R_z = 0$, the additions leave every R_j unchanged, so (I1) and (I2) hold; the removed slot had $R_z = 0$ and owned no pivot. *Case B.* Every R_j with $j \in O$ is non-zero, and by (I2) the lows are distinct, so p is well defined and $\operatorname{low}(R_p)$ is strictly smaller than $\operatorname{low}(R_j)$ for $j \in O \setminus \{p\}$. All cells of R_p have keys at most $\operatorname{ord}(\operatorname{low}(R_p)) < \operatorname{ord}(\operatorname{low}(R_j))$, so $\operatorname{low}(R_j) \notin R_p$ and $\operatorname{low}(R_j + R_p) = \operatorname{low}(R_j)$; in particular $R_j + R_p \neq 0$. Lows are unchanged and the low of R_p is released with slot p, so (I2) holds; (I1) holds by linearity. (The minimal choice of p is what guarantees this: adding an R_p with a larger low would move lows and require further reduction.) For B_{k+1}: σ has no coface, so no $R_i = \partial V_i$ of dimension $k+1$ contains σ and σ is not a pivot there; deleting the zero row σ leaves (I1)–(I3) intact.

(d) By (a), r_k increases by one exactly when the inserted σ is negative, and decreases by one exactly in Case B; n_k changes by ± 1; r_{k+1} is unchanged because only a zero row of B_{k+1} is added or removed (Observation 2.1); all other n_i and r_i are unchanged. Formula (1) gives the stated changes. If $z \in Z$, then V_z is a cycle containing σ. Conversely, if a cycle $c \in Z_k(K)$ contains σ, write c in the basis $\{V_j : R_j = 0\}$ of $Z_k(K)$ from (a); some V_j in this expansion contains σ, and that j lies in Z. For insertion, if σ is positive then the final V_j is a cycle of K' containing σ. Conversely, if a cycle c of K' contains σ, then $\partial\sigma = \partial(c + \sigma)$ lies in the column space of $B_k(K)$, so r_k does not increase, and by (a) the new slot has $R_j = 0$. □

Vertices need no slots: inserting a vertex increases n_0 and β_0 by one, and a deletable vertex is isolated, so deleting it decreases both by one (Observation 2.1 and (1)). A batch is processed one atomic edit at a time, so Theorem 1 makes the Betti numbers exact after every update. The choice of z in Case A is free; it only affects sparsity. Theorem 1 holds for every key function, so the order chosen in Section 4 is a matter of cost alone.

Initialisation and rebuilds. The DRB is initialised, and periodically rebuilt, by the static reduction with clearing [9], tracking V: for $k = d, d-1, \ldots, 1$, process the columns of B_k in increasing key order; if the k-simplex τ is the low of a non-zero reduced column R of B_{k+1}, *clear* it ($R_\tau := 0$, $V_\tau := R$); otherwise reduce it as in steps 2–3 of INSERT, starting from $V_\tau = \{\tau\}$, $R_\tau = \partial\tau$.

Proposition 1.1 (Rebuild). *For every injective key function, the static reduction with clearing produces slots satisfying (I1)–(I3) in every dimension. Moreover, every V_τ consists of τ and cells with smaller keys. The same holds for the reduction without clearing. In particular the ranks it reports are exact even when V is not stored.*

Proof. We show $\mathrm{low}(V_\tau) = \tau$ for every column τ, by induction along the processing order. A non-cleared τ ends with $V_\tau = \{\tau\} + \sum V_p$ over pivot owners p processed earlier, hence with smaller keys, and by induction every cell of V_p has key at most $\mathrm{ord}(p)$. A cleared τ has $V_\tau = R$ with $\mathrm{low}(R) = \tau$. Ordering rows and columns by key, V is therefore upper unitriangular, hence invertible: (I3). (I1) holds for non-cleared columns by linearity, and for cleared ones because $R = \partial V'$ for the slot V' of B_{k+1} that produced it (induction over dimensions), so $\partial V_\tau = \partial\partial V' = 0 = R_\tau$. (I2): a non-cleared column ends with $R_\tau = 0$ or with a low that no earlier column owns; cleared columns are zero. Without clearing every column is of the non-cleared kind and the same argument applies. The last statement follows from Theorem 1(a). □

Note that the correctness of clearing here follows from the invariants alone; we do not need the classical fact that a cleared column would have reduced to zero. After deletions, however, V is no longer triangular (Case A adds V_z, whose low may exceed that of V_j), so the DRB maintains ranks and a cycle basis but not a persistence pairing, and we claim none. This is the difference from vineyard updates [10], which keep V triangular for a changing order.

Rebuild policy. mmhm rebuilds with the Morse order of Section 4; it rebuilds after update t if $t - t_{\mathrm{last}} \geq m$ or $\mathrm{nnz}(R) + \mathrm{nnz}(V) > (1+\alpha)\max(\mathrm{nnz}_{\mathrm{last}}, |K|)$, where $\mathrm{nnz}_{\mathrm{last}}$ is the value right after the previous rebuild (defaults $m = \infty$, $\alpha = 1$), and the rebuild is charged to update t. drb-id uses insertion keys and never rebuilds; mmhm-norb uses the Morse order at the initial build only; drb-id-rb applies the same policy as mmhm but rebuilds in insertion order (Section 6.2).

4 The Morse order

Discrete Morse theory [21] enters only through the choice of keys at a rebuild. A rebuild computes an acyclic matching by collapses, in the spirit of [5], and orders cells by reverse removal time.

Definition 4.1 (Collapse matching and Morse order)**.** Run the following on $P := K$, with a counter $t := 0$ and a queue Q of the cells x that have exactly one immediate coface in P (*free faces*).

1. While $Q \neq \emptyset$: pop x; skip it unless $x \in P$ and x has exactly one immediate coface y in P. Then *pair* (x, y), set $\mathrm{rm}(y) := t$, $\mathrm{rm}(x) := t+1$, $t \leftarrow t+2$, remove y and x from P, and push every facet of y or x that has become free.
2. If $P \neq \emptyset$, choose a cell c of the highest dimension present in P (smallest vertex tuple), declare it *critical*, set $\mathrm{rm}(c) := t$, $t \leftarrow t+1$, remove it, push the facets that have become free, and go to 1.

The *Morse order* is $\mathrm{ord}(x) := -\mathrm{rm}(x)$ (relabelled to $0, \ldots, |K|-1$). Write M for the set of pairs, Crit for the critical cells, Low for the cells x with $(x, y) \in M$ and Up for the cells y with $(x, y) \in M$; for $x \in \mathrm{Low}$ let $\mu(x)$ be its partner. For a cell x, its *youngest facet* is the facet with the largest key and its *oldest coface* the coface with the smallest key.

The removed cells are always maximal in P, and P remains a subcomplex. For the paired cell y this uses the simplicial diamond property: if y had a coface $w \in P$, then w would have exactly two facets containing x, namely y and some $y' \neq y$, and $y' \in P$ because P is a subcomplex, so x would not be free. The queue discipline is not specified; all results below hold for every discipline.

A *gradient path* (V-path) is a sequence $\tau_0, \sigma_0, \tau_1, \sigma_1, \ldots, \tau_m$ with $(\tau_i, \sigma_i) \in M$ and τ_{i+1} a facet of σ_i different from τ_i.

Lemma 2 (Gradient pairs are apparent pairs)**.** *For every execution of the collapse procedure, with* $\mathrm{ord} = -\mathrm{rm}$*:*

(a) ord *is a filtration order:* $\mathrm{ord}(f) < \mathrm{ord}(s)$ *whenever* f *is a facet of* s*.*
(b) rm *strictly increases along the lower cells of every gradient path* ($\mathrm{rm}(\tau_{i+1}) > \mathrm{rm}(\tau_i)$)*; in particular* M *has no closed gradient path, so it is an acyclic matching.*
(c) For every $(\tau, \sigma) \in M$*:* τ *is the youngest facet of* σ*, i.e.* $\tau = \mathrm{low}(\partial\sigma)$*, and* σ *is the oldest coface of* τ*.*
(d) If $z \neq 0$ *is a cycle, then* $\mathrm{low}(z) \notin \mathrm{Up}$*.*
(e) In the static reduction under ord*, with or without clearing, for every* $(\tau, \sigma) \in M$ *the column* σ *is not cleared, receives no addition, and obtains pivot* τ*; thus* $R_\sigma = \partial\sigma$ *and* (τ, σ) *is an* apparent pair *[1].*

Proof. (a) Only maximal cells of P are removed (above), so every cell is removed before each of its facets.

(b) When σ_i is removed at time $\mathrm{rm}(\sigma_i)$, its facet τ_{i+1} is still in P. Times $\mathrm{rm}(\sigma_i)$ and $\mathrm{rm}(\tau_i) = \mathrm{rm}(\sigma_i) + 1$ are used by σ_i and τ_i, and $\tau_{i+1} \neq \tau_i$, so $\mathrm{rm}(\tau_{i+1}) > \mathrm{rm}(\tau_i)$. A closed gradient path would give $\mathrm{rm}(\tau_0) < \mathrm{rm}(\tau_0)$.

(c) The pair is removed at consecutive times, so $\mathrm{ord}(\tau) = \mathrm{ord}(\sigma) - 1$: the two cells are adjacent in the order. By (a) every facet of σ precedes σ, hence has key at most $\mathrm{ord}(\tau)$, and every coface of τ follows τ, hence has key at least $\mathrm{ord}(\sigma)$. (This is the elementary fact that a facet and a coface that are adjacent in a filtration order form an apparent pair.)

(d) Let $u = \mathrm{low}(z) \in \mathrm{Up}$ with $u = \mu(w)$. The coefficient of w in $\partial z = 0$ vanishes and $w \in \partial u$, so some $u' \in z$ with $u' \neq u$ has w as a facet. By (c), u is the oldest coface of w, so $\mathrm{ord}(u') > \mathrm{ord}(u)$, contradicting $u = \mathrm{low}(z)$.

(e) If σ were cleared, it would be the low of a reduced column R of the next dimension; $R = \partial V'$ (Proposition 1.1) is a non-zero cycle with low $\sigma \in \mathrm{Up}$, contradicting (d). Columns are processed in increasing key order. When σ is processed, every earlier column x has $R_x = 0$ (if cleared) or $R_x = \partial V_x$ with all cells of V_x of key at most $\mathrm{ord}(x) < \mathrm{ord}(\sigma)$ (Proposition 1.1); by (c) no such cell has τ as a facet, so $\tau \notin R_x$ and $\mathrm{pivot}(\tau)$ is undefined. Since $\mathrm{low}(\partial\sigma) = \tau$ by (c), the reduction loop performs no addition and assigns $\mathrm{pivot}(\tau) = \sigma$. □

Corollary 2.1 (Work happens only on critical columns)**.** *In the static reduction with clearing under the Morse order, the column of every $x \in \mathrm{Low}$ with $\dim x \geq 1$ is cleared, the column of every $y \in \mathrm{Up}$ is apparent, and therefore every column addition is performed while reducing the column of a critical cell.*

Proof. For $(x, y) \in M$ with $\dim y = k + 1 \geq 2$, Lemma 2(e) applied to B_{k+1}, which is processed before B_k, gives $R_y = \partial y$ with $\mathrm{low}(R_y) = x$, so x is cleared. Upper cells are apparent by Lemma 2(e). Every remaining column belongs to a critical cell. □

Morse boundary. By Lemma 2(b) the following recursion is well founded and defines a linear map $\Pi\colon C_{k-1} \to C_{k-1}$: $\Pi(\tau) = \tau$ for $\tau \in \mathrm{Crit}$, $\Pi(\tau) = 0$ for $\tau \in \mathrm{Up}$, and $\Pi(\tau) = \sum_{\tau' \in \partial\mu(\tau),\, \tau' \neq \tau} \Pi(\tau')$ for $\tau \in \mathrm{Low}$. Thus $\Pi(\tau)$ is the sum, over all gradient paths from τ to a critical cell, of their endpoints, counted mod 2. The *Morse boundary* of a critical k-cell c is $\partial^M c := \Pi(\partial c)$, the boundary of the Morse complex of the matching [21]; the Morse boundary matrices B_k^M have rows $\mathrm{Crit} \cap K_{k-1}$ and columns $\mathrm{Crit} \cap K_k$. For $y \in \mathrm{Up}$ with partner x, the definition of $\Pi(x)$ gives

$$\Pi(\partial y) = \Pi(x) + \sum_{f \in \partial y,\, f \neq x} \Pi(f) = 0. \tag{2}$$

Corollary 2.2 (Critical columns perform the Morse reduction)**.** *Consider the static reduction with clearing under the Morse order and, next to it, the standard reduction with clearing of the Morse boundary matrices $B_d^M, \ldots, B_1^M$, critical cells taken in increasing key order. For every critical k-cell c, $k \geq 1$:*

- *(i) the additions performed on column c are first additions of apparent columns $R_y = \partial y$ ($y \in \mathrm{Up}$), then additions of columns of critical cells;*
- *(ii) if c is not cleared, then when the first phase ends the column contains no cell of* Low *and its restriction to* Crit *equals $\partial^M c$;*
- *(iii) the second phase performs, one for one, the additions that the Morse reduction performs on column c; c is cleared exactly when it is cleared in the Morse reduction; and the final column R_c contains no cell of* Low*, while $R_c \cap \mathrm{Crit}$ is the reduced Morse column of c.*

Each first-phase addition $R \leftarrow R + \partial y$ with $\mathrm{low}(R) = x$ and $y = \mu(x)$ replaces x by the other facets of y; it is one step of the discrete gradient flow, and $\Pi(R)$ does not change by (2). This is the precise sense in which the reduced column of a critical cell "traces its gradient flow". R_c may still contain cells of Up; we observed this in the randomised checks and claim nothing about them.

Proof. We use the rule of Definition 4.1 that a critical cell is chosen of the highest dimension present in P. (Lemma 2 did not use it.)

Claim (separation). For every q, every critical q-cell has a smaller key than every q-cell of Low. Indeed, when a critical q-cell t is removed, P contains no $(q+1)$-cell. If $\ell \in \mathrm{Low}$ were a q-cell with $\mathrm{rm}(\ell) > \mathrm{rm}(t)$, its partner $\mu(\ell)$, a $(q+1)$-cell removed at time $\mathrm{rm}(\ell) - 1 \geq \mathrm{rm}(t)$, would be in P at that moment.

We argue by induction over the dimensions (top-down) and, within B_k, over the columns in increasing key order. The statement about clearing is shown below; a cleared column receives no addition and has $R_c = 0$, matching the zero column of the Morse reduction. So let c be a column that is not cleared, and let R denote its column during its reduction; initially $R = \partial c$.

First phase. Suppose $\ell = \mathrm{low}(R) \in \mathrm{Low}$ and let $y = \mu(\ell)$. All cells added to V_c so far are c and upper cells with smaller keys, so ℓ is a facet of a cell x with $\mathrm{ord}(x) \leq \mathrm{ord}(c)$; since y is the oldest coface of ℓ (Lemma 2(c)) and

$y \neq c$, $\mathrm{ord}(y) < \mathrm{ord}(c)$. By Lemma 2(e), the column of y has been processed, holds $R_y = \partial y$ and owns $\mathrm{pivot}(\ell)$; the algorithm adds it, which strictly decreases the low and preserves $\Pi(R)$ by (2). The phase ends with $R = 0$ or with $t = \mathrm{low}(R) \notin \mathrm{Low}$. In the latter case R is a non-zero cycle (by (I1), $R = \partial V_c$), so $t \notin \mathrm{Up}$ by Lemma 2(d), and $t \in \mathrm{Crit}$. By the separation claim every cell of R, having key at most $\mathrm{ord}(t)$, lies outside Low. As Π fixes critical cells and kills upper cells, $R \cap \mathrm{Crit} = \Pi(R) = \Pi(\partial c) = \partial^M c$. This proves (ii) and the first part of (i).

Clearing. The cells cleared in B_k are the lows of the non-zero reduced columns of B_{k+1}: the lower cells (lows of apparent columns) and the lows of non-zero critical columns. By the induction hypothesis (iii) for dimension $k+1$, the latter are critical and coincide with the lows of the non-zero reduced Morse columns, i.e. with the cells cleared in the Morse reduction. So a critical c is cleared in one reduction if and only if in the other; in the top dimension nothing is cleared in either.

Second phase. A pivot on a critical row cannot be owned by an apparent column (its low is in Low) or a cleared one (zero), so it is owned by a critical column c' processed earlier. By the induction hypothesis $R_{c'}$ has no cell of Low, it is a cycle, its low is critical and equals $\mathrm{low}(R_{c'} \cap \mathrm{Crit})$, and $R_{c'} \cap \mathrm{Crit}$ is the reduced Morse column of c'; hence the pivot tables of the two reductions agree on critical rows. Adding such columns keeps $R \cap \mathrm{Low} = \emptyset$ and keeps R a cycle, so while $R \neq 0$ its low is critical and equals $\mathrm{low}(R \cap \mathrm{Crit})$. The sequence of pivot look-ups and additions on $R \cap \mathrm{Crit}$, which starts from $\partial^M c$, is therefore exactly the Morse reduction of column c, and no apparent column is added any more. This proves (i) and (iii). □

Remark 4.2 (Relation to known results). Lemma 2 is not new in substance. Its part (c) is the elementary fact noted in its proof, and the relation between gradient pairs and apparent pairs is known in both directions: the apparent pairs of any simplexwise filtration form a discrete gradient, which Ripser uses to skip columns [1], and Bauer and Roll [2] relate the gradient pairs of a (generalised) discrete Morse function to the zero-persistence apparent pairs of a lexicographically refined filtration order. Lemma 2 is a self-contained version for the order produced by our collapse procedure, in the form used below: for every queue discipline, with and without clearing, and with part (d), on which Corollary 2.2 rests. Mischaikow and Nanda [32] showed that Morse reductions compatible with a filtration preserve persistent homology; Corollary 2.2 is a matrix-level form of this for our order: on critical rows and columns, the static reduction *is* the reduction of the Morse complex, and by Corollary 2.1 the other columns cost no additions. This is what the full-morse baseline exploits. In the language of algebraic Morse theory [26, 38], the first phase of Corollary 2.2 is Gaussian elimination of the invertible entries $\langle \partial y, x \rangle = 1$ of the matched pairs, which is how the Morse complex arises algebraically; the order makes the reduction algorithm perform these eliminations by itself, without a separate Morse-reduction pass.

Remark 4.3 (Scope). Lemma 2 and its corollaries describe the state produced by a rebuild. Between rebuilds the matching is not maintained, inserted cells get keys larger than all others, and deletions add chains whose lows may exceed those of their targets. We make no claim that the DRB stays sparse between rebuilds; the Morse order is a heuristic for small footprints and short reduction chains, its effect is measured, and the rebuild policy bounds the drift of $\mathrm{nnz}(R) + \mathrm{nnz}(V)$.

5 Update cost and the limits of locality

5.1 Output-sensitive cost

We state the cost in two storage models. In the *ordered-set model*, a chain is a balanced search tree keyed by ord and a footprint is a set of slots; inserting, removing or testing one cell or slot, one pivot-table operation and one low query are *elementary operations*, each of $O(\log n)$ time, $n = |K|$. In the *sorted-array model*, which is what the engine implements (Section 6.1), a chain is an increasing array of keys, its low is its last entry, and a sum $x + y$ of two chains is a linear merge that costs $\Theta(|x| + |y|)$, the size of the target included. For an insertion we write $R^{(i)}$ and $V^{(i)}$ for the working column just before the i-th addition of its reduction chain.

Proposition 3 (Output-sensitive cost)**.** *Let σ be a k-simplex, $k \geq 1$.*

- *(a)* INSERT(σ) *with reduction chain $p_1, \ldots, p_m$ costs $\Theta\big(k + 1 + \sum_{i=1}^{m}(1 + |R_{p_i}| + |V_{p_i}|)\big)$ elementary operations in the ordered-set model and $\Theta\big(k + 1 + \sum_{i=1}^{m}(1 + |R^{(i)}| + |V^{(i)}| + |R_{p_i}| + |V_{p_i}|)\big)$ time in the sorted-array model. In both, the p_i are pairwise distinct, so $m \leq r_k$ and $\sum_i(|R_{p_i}| + |V_{p_i}|) \leq \mathrm{nnz}(R) + \mathrm{nnz}(V)$ of the DRB of B_k.*
- *(b)* DELETE(σ) *with footprint $O = \mathrm{occ}(\sigma)$ costs, including the maintenance of the footprints and of* pivot, *$\Theta\big(|O|\,(1 + |V_z|)\big)$ elementary operations in Case A and $\Theta\big(|O|\,(1 + |R_p| + |V_p|)\big)$ in Case B in the ordered-set model, and $\Theta\big(\sum_{j \in O}(1 + |V_j| + |V_z|)\big)$ in Case A and $\Theta\big(\sum_{j \in O}(1 + |R_j| + |V_j| + |R_p| + |V_p|)\big)$ in Case B in the sorted-array model.*

(c) An update costs the sum over its atomic edits; a vertex edit costs $O(1)$.

Proof. (a) Each addition touches every cell of R_p and V_p once, and in the sorted-array model also every cell of the working column, and performs one pivot look-up and one low query; the final chain V_j, of size at most $1 + \sum_i |V_{p_i}|$, is entered into the footprints; $|\partial\sigma| = k + 1$. Along the chain the lows strictly decrease (proof of Theorem 1(b)), and each slot owns one low, so no slot is added twice. (b) Scanning O to determine Z, and z or p, costs $O(|O|)$ (sizes $|V_j|$ are stored). In Case A each of the $|O| - 1$ additions touches the $|V_z|$ cells of V_z in V_j and in the footprints (in the sorted-array model also the $|V_j|$ cells of the target), and removing slot z removes it from $|V_z|$ footprints. Case B is the same with R_p and V_p, plus one pivot removal. □

The engine is a hybrid: when short chains are added to a long working column it switches the column to a max-heap, which brings an insertion to the ordered-set bound up to a logarithmic factor, whereas its deletions pay the sorted-array cost (Section 6.1). The engine does not log these costs. Per update it records the number of column additions (the number m of terms in (a), and $|O| - 1$ per deletion in (b)), the sum of the footprints $|\operatorname{occ}(\sigma)|$ of the deleted simplices, and $\operatorname{nnz}(R) + \operatorname{nnz}(V)$ (Section 6.4). These are proxies: they count the terms of the sums above but not the sizes of the chains involved, so they need not move in the same direction as running time. Neither model counts the engine's lazy footprint lists: entries left behind by earlier eliminations are skipped when $\operatorname{occ}(\sigma)$ is read, which adds work proportional to the stale entries met.

The quantities in Proposition 3 — footprints $|\operatorname{occ}(\sigma)|$, reduction-chain lengths and the sizes of the chains involved — depend on the whole history and on the order, not on s_t. We report proxies for them as measurements and state no locality theorem. Proposition 4 shows that none is possible for this data structure.

5.2 No worst-case locality

Proposition 4 (No worst-case locality for the DRB)**.** *Let K be either (W5a) the cycle graph C_L, $L \geq 3$, and σ one of its edges, with $n := L$; or (W5b) a triangulated 2-sphere (more generally, a connected closed triangulated surface) and σ one of its triangles, with $n := n_2$. Let $k = \dim \sigma$. Then, for every key function and every choice in Case A:*

- *(a) for every DRB of $B_k(K)$ satisfying (I1)–(I3), DELETE(σ) is in Case A with $|V_z| = n$ and performs at least n elementary operations;*
- *(b) for every DRB of $B_k(K \setminus \{\sigma\})$ satisfying (I1)–(I3), INSERT(σ) is positive, ends with $V_j = K_k$, and its reduction chain satisfies $\sum_i |V_{p_i}| \geq n - 1$;*
- *(c) if S is any DRB state of $K \setminus \{\sigma\}$ and S' is the state after INSERT(σ), then DELETE(σ) applied to S' restores S exactly. Hence, after the first deletion, toggling σ alternates between two fixed states S_0, S_1, and each toggle costs at least n and, in the ordered-set model, at most $O\big(n + \operatorname{nnz}(R) + \operatorname{nnz}(V)\big)$ elementary operations, with* nnz *taken in S_0.*

So on these families every toggle of σ, an update of size $s_t = 1$, costs this algorithm $\Omega(n)$ work in either storage model, and $\Theta(n)$ elementary operations whenever $\operatorname{nnz}(R) + \operatorname{nnz}(V) = O(n)$ in S_0. Other updates of size one, such as inserting a pendant edge, can of course be cheap.

The statement bounds the work of *this* algorithm, which must maintain (I1)–(I3) and the footprint index; it is not a lower bound for computing Betti numbers under edits. For (W5a), $\beta_1 = n_1 - n_0 + \beta_0$ and β_0 can be maintained by dynamic connectivity in polylogarithmic amortised time per edge edit [25]; for (W5b), see Remark 5.1. What the proposition does show is that any method that stores cycle representatives explicitly, as (I3) forces, has to write the n-cell cycle when it appears.

Proof. In both cases $Z_k(K) = \{0, K_k\}$. For C_L, a 1-cycle is a set of edges meeting every vertex an even number of times, and the only non-empty such set is the whole cycle. For a connected closed surface, a 2-cycle that contains a triangle contains, for each edge of it, the other triangle on that edge; as the surface is strongly connected, it contains every triangle. Also $Z_k(K \setminus \{\sigma\}) = \{0\}$, since the only non-zero cycle of K contains σ.

(a) By Theorem 1(a) exactly one slot z has $R_z = 0$, and $V_z = K_k \ni \sigma$. So $z \in O$, the deletion is in Case A with $Z = \{z\}$, and removing slot z removes it from the footprints of all n cells of V_z.

(b) $\beta_k(K) = \beta_k(K \setminus \{\sigma\}) + 1$, so by Theorem 1(d) σ is positive; the final V_j is then a non-zero cycle of K, i.e. $V_j = K_k$. Since $V_j = \{\sigma\} + \sum_i V_{p_i}$, we get $n \leq 1 + \sum_i |V_{p_i}|$.

(c) In S no chain contains σ, and INSERT(σ) creates one slot j and modifies no other. So in S' we have $\operatorname{occ}(\sigma) = \{j\}$ and $R_j = 0$ by (b): DELETE(σ) is in Case A with $O = Z = \{j\}$, modifies nothing and removes j. The key given to

σ differs between insertions, but it is always the largest key in use and the algorithms only compare keys, so every insertion from S_0 performs the same computation. The bounds follow from (a), (b) and Proposition 3. □

Remark 5.1 (Surfaces need no linear algebra). Let S be a connected triangulated surface, possibly with boundary (every edge lies in one or two triangles and every vertex link is a path or a cycle). Over $\mathbb{Z}_2$: $\beta_0 = 1$; $\beta_2 = 1$ if no edge lies in exactly one triangle and $\beta_2 = 0$ otherwise (by strong connectivity a non-zero 2-cycle is the set of all triangles, whose boundary is the set of boundary edges); and $\beta_1 = \beta_0 + \beta_2 - \chi(S)$ with $\chi = n_0 - n_1 + n_2$. Under atomic edits that keep S a connected surface, n_0, n_1, n_2 and the number of boundary edges are maintained in $O(1)$ per edit from per-edge triangle counts. All Betti numbers are then available in $O(1)$ per edit without any matrix — provided that connectivity and the surface property are known in advance; the counters do not certify connectivity. This is the euler method. Consequently, workloads consisting of connected surfaces with known connectivity cannot evaluate the algebraic part of a homology-maintenance method: a method that switches to these formulas returns correct Betti numbers there whether or not its matrix updates are correct.

6 Experiments

The experiments answer nine questions, E0 to E8 (Table 2): whether the exact methods are exact (E0); how the cost of an update depends on the size of the complex and on the size of the update (E1, E2); how the methods behave on genuine three-dimensional topology changes, on non-manifold complexes and on surfaces (E3–E5); where locality fails (E6); how much the rebuild policy matters (E7); and what the Morse order buys (E8). Sections 6.1 to 6.5 describe the setup and Sections 6.6 to 6.14 the results. Every measured number, table and figure is produced by a script from the raw per-update records of the runs (`experiments/README.md` describes how to regenerate them); none is typed by hand.

6.1 Implementation and platform

Engine. All methods except gudhi run in one C++17 program and share its complex store, which maps the sorted vertex tuple of a simplex to its facets and cofaces and validates every atomic edit. Order keys are dense 32-bit integers; a (re)build assigns $0, \ldots, N-1$ in the chosen order, and every later insertion takes the next unused key (Section 3). A chain is a strictly increasing array of keys, so its low is the last entry and a $\mathbb{Z}_2$ addition is a linear merge; a working column that receives many short chains switches to a max-heap with lazy cancellation. Deletions therefore pay the sorted-array cost of Proposition 3, and insertions its ordered-set cost up to a logarithmic factor. The footprint index keeps the exact count $|\operatorname{occ}(\sigma)|$ and a lazily filtered list of slots. In Case A the engine chooses the cycle V_z with the fewest cells (ties: the smallest slot identifier). Only the methods that may compute a Morse order after K_0 keep an index of the cells by vertex tuple; among the DRB methods that is mmhm. The collapse of Definition 4.1 uses a first-in-first-out queue initialised in (dimension, vertex tuple) order, so the matching depends only on the complex. Debug builds check (I1)–(I3), the pivot table and the footprint index after every operation.

Platform. All runs use one cloud virtual machine with an Intel Xeon processor at 2.10 GHz (4 cores, one thread per core; 48 KiB L1 data and 2 MiB L2 cache per core, 260 MiB shared L3 cache) and 15 GiB of memory, running Ubuntu 24.04.5 LTS with Linux 6.18. The engine is compiled by g++ 13.3.0 (CMake 3.28.3) with `-O2 -DNDEBUG` and no machine-specific tuning. The GUDHI baseline and oracle run under Python 3.13.16 with GUDHI 3.13.0 and numpy 2.5.3. Every method runs single-threaded.

6.2 Methods

Table 1 lists the methods; the names are those used in the code, the raw data and the figures.

Static baselines. full-id, full-sorted and full-morse recompute all ranks after every update by the static reduction with clearing (Section 3), without storing V: full-id in insertion order, full-sorted in the canonical order, full-morse in the Morse order of a collapse matching computed anew after every update. By Corollaries 2.1 and 2.2 the column additions of full-morse all occur in critical columns and, after the gradient-flow steps, are those of the reduction of the Morse complex, but it pays for a collapse after every update. The insertion order of full-id is the order in which the cells now present were inserted, so it depends on the history of edits: refilled cells receive late keys (Section 6.4 reports the consequence). The canonical order of full-sorted depends only on the current complex, so it cannot drift with the history; it is our history-independent reference. All three share the store and the reduction code with the dynamic methods, so differences reflect mostly the algorithms; the remaining implementation differences are this history dependence and the index of cells by vertex tuple that full-sorted and mmhm maintain (E4). gudhi replays the same edits on a GUDHI simplex tree [30] through its Python interface and, after every update, calls `compute_persistence` with the

Table 1: Methods. *Order*: the key order of the cells; *canonical*: by dimension, then lexicographically by vertex tuple. V, occ: whether the change of basis and the footprint index are stored. *Rebuilds*: when the structure is recomputed; "policy" is the rebuild policy of Section 3. *Exact*: whether the reported Betti numbers are correct after every update. All methods except gudhi run in the same engine and share its complex store.

Method	Order	V, occ	Rebuilds	Exact	Role
full-id	insertion	no	every update	yes	static recomputation with clearing, ranks only
full-sorted	canonical	no	every update	yes	static recomputation in an order that does not depend on the history
full-morse	Morse, recomputed	no	every update	yes	static recomputation in the Morse order of a new collapse matching
gudhi	GUDHI's own	—	every update	yes	external static baseline: GUDHI simplex tree, persistence over $\mathbb{Z}_2$
drb-id	insertion	yes	never	yes	ablation: neither Morse order nor rebuilds
drb-id-rb	insertion	yes	policy	yes	ablation: rebuilds without the Morse order
mmhm	Morse	yes	policy	yes	the method of Sections 3 and 4
mmhm-norb	Morse of K_0	yes	never	yes	ablation: no rebuilds
drb-rand	random	yes	never	yes	ablation: an unstructured order
euler	—	—	—	on surfaces	counters of Remark 5.1; surfaces only

arguments `homology_coeff_field=2` and `persistence_dim_max=True`, and then `betti_numbers`; without the second argument GUDHI omits the top dimension. GUDHI computes persistence pairs, a richer output, and is not designed for this task; it serves as a reference point for a widely used library and as an oracle.

Dynamic methods. mmhm is the method of Sections 3 and 4: a dynamic reduced basis built in the Morse order and rebuilt in a fresh Morse order under the rebuild policy of Section 3, with $m = \infty$ and $\alpha = 1$ unless stated otherwise (E7 varies both). Three ablations separate its ingredients. drb-id uses insertion keys and never rebuilds. mmhm-norb uses the Morse order of K_0 and never rebuilds. drb-id-rb rebuilds under the same policy and parameters as mmhm, but its rebuild reruns the static reduction with clearing in the current insertion order (the keys in use, renumbered) instead of computing a new matching. So mmhm against drb-id-rb isolates the order used at a rebuild, and mmhm against mmhm-norb isolates rebuilding. drb-rand builds the DRB in a uniformly random order of the cells of each dimension, drawn from the seed, and never rebuilds; against drb-id and mmhm-norb it tests whether a structured order matters at all (E8).

Surface counters. euler maintains χ and the numbers of edges with one, two or more cofaces in $O(1)$ per atomic edit and returns the Betti numbers of Remark 5.1. At the end of every update it checks that the complex has a triangle, no tetrahedron, and no edge outside one or two triangles, and aborts otherwise; connectivity and the absence of pinched vertices are assumed, not checked.

6.3 Workloads

Five generators produce the update sequences. Each is deterministic given its parameters and seed, emits only valid atomic edits (checked again by an independent validator), lists deletions top-down and insertions bottom-up, and records its parameters, the sizes of all states and the number of edits per update. For W1, W2, W3 and W5 the Betti numbers of every state are known in closed form and are written next to the sequence. Distances are graph distances in the base mesh.

W1: punctured surfaces. The base surface is the geodesic icosphere of frequency $f = 2^L$, with $10f^2 + 2$ vertices, $30f^2$ edges and $20f^2$ triangles, or the $n \times m$ periodic grid torus with every square split along one diagonal, with nm vertices, $3nm$ edges and $2nm$ triangles. For a vertex v, the patch $P(v, k)$ is the set of triangles incident to a vertex at distance at most $k - 1$ from v; we use $k = 2$. When v and its neighbours have degree six, $P(v, 2)$ has 24 triangles and opening it takes 61 atomic edits; no vertex of our surfaces has a larger degree, so no update has more edits. A patch around a random vertex is used only if its closure is a disk and its vertices are at distance more than one from the vertices of every existing hole. Opening a hole deletes the triangles of the patch, then its edges, then the vertices left without cofaces; closing a hole is the exact reverse of its opening. With h holes, the generator opens a hole if $h = 0$; otherwise it opens one with probability 0.75, 0.5 or 0.25 when h is below, at or above the target 3, and closes one otherwise or when no valid patch is found. The Betti numbers are $(1, \max(h - 1, 0), [h = 0], 0)$ for the sphere and $(1, 2 + \max(h - 1, 0), [h = 0], 0)$ for the torus, where $[h = 0]$ is 1 if $h = 0$ and 0 otherwise. W1 is the setting of Remark 5.1, which E5 examines. It also contains the phenomenon of Proposition 4: when no hole is open, the surface

is a connected closed surface, so the deletion that opens the first hole costs every DRB method $\Omega(n_2)$ operations by Proposition 4(a), and the last triangle inserted when the last hole closes costs $\Omega(n_2)$ by Proposition 4(b).

W2: refinement and coarsening in a 3-ball. The base complex is the Freudenthal (Kuhn) triangulation of a grid of $N \times N \times N$ cubes, each split into six tetrahedra; it has $(N+1)^3$ vertices and about $26N^3$ simplices. A refinement applies s stellar $1 \to 4$ subdivisions to s tetrahedra collected breadth-first over face adjacency from a uniformly random tetrahedron; tetrahedra created by earlier subdivisions may be subdivided again. A coarsening undoes s leaf subdivisions, those whose four tetrahedra are all still present: a uniformly random one and the $s-1$ nearest to it, measured by the Euclidean distance between the inserted vertices. A subdivision and its inverse each consist of 16 atomic edits, so $s_t = 16s$. Because coarsening can only undo subdivisions, K_0 already contains $8s$ live subdivisions, and with x live subdivisions the generator refines with probability $\min\big(0.95, \max(0.05,\ 0.5 - 0.5(x - 8s)/(8s))\big)$, and always when fewer than s leaf subdivisions exist. Since $|K| = |K_{\text{base}}| + 14x$, the size performs a random walk around its initial value. The Betti numbers are $(1, 0, 0, 0)$ after every update, but not within an update: by Theorem 1(d) every atomic edit changes one Betti number by one.

W3: cavities in a 3-ball. The base complex is that of W2. A centre v is drawn uniformly among the interior vertices, and the ball $B(v, k)$ is the set of tetrahedra incident to a vertex at distance at most $k-1$ from v; we use $k = 2$. Carving deletes the tetrahedra of B and then, top-down, every face left without cofaces. A ball is accepted only if it is more than one ring away from every carved ball, its boundary is a 2-sphere, and it meets the boundary of the cube in no triangle (a cavity: β_2 grows by one) or in one disk (a dent: no change); other contacts, which could bore a tunnel, are rejected. Refilling is the exact reverse of a carving. The number of carved balls follows the same walk as the number of holes in W1, with target 4. The Betti numbers are $(1, 0, c, 0)$, where c is the number of cavities.

W4: flag complexes of moving points. n points are drawn uniformly in the unit square or the unit cube ("2D" and "3D" in the tables and figures). A radius ε is calibrated on the initial points so that the ε-neighbourhood graph has exactly $5n$ edges (mean degree 10), and K is the flag complex of this graph truncated at dimension $D \in \{2, 3\}$. An update moves one uniformly random point by adding to each coordinate an independent offset drawn uniformly from $[-\varepsilon/4, \varepsilon/4]$, reflected at the border of the domain, and keeps its vertex identifier: it deletes the closed star of the vertex top-down and inserts the vertex and its new star bottom-up. The complexes are not manifolds and all Betti numbers vary; they are not known in closed form and are checked against the oracles only.

Caveat: truncation. In a flag complex truncated at dimension D, the vertices of a clique of size $D+2$ span a $(D+1)$-simplex that is not in K, so its boundary is a D-cycle that is never filled. Hence β_D is large: at INIT, β_D/n lies between 2.8 and 6.2 in our W4 instances (Table 12). This is genuine linear algebra on a non-manifold complex, but β_D is dominated by the truncation, not by the shape of the point cloud. We use W4 as an algebraic stress test and draw no conclusion about the topology of point clouds from it.

W5: non-locality witnesses. (a) The cycle graph C_L, whose updates toggle the edge $\{0, 1\}$: odd-numbered updates delete it and even-numbered ones insert it again. (b) The icosphere of frequency 2^L, whose updates toggle one triangle chosen from the seed in the same way. By Proposition 4, every toggle costs every DRB method $\Omega(L)$, respectively $\Omega(n_2)$, operations; once the states alternate, a toggle costs $O(L + \text{nnz}(R) + \text{nnz}(V))$, respectively $O(n_2 + \text{nnz}(R) + \text{nnz}(V))$. euler needs $O(1)$ per toggle on (b) and does not apply to (a), which is a graph and not a surface.

Sweeps. Table 2 lists the workloads, parameters and methods of each experiment. Every timed configuration ran with 3 seeds.

6.4 Timing protocol

The input sequence is parsed completely before timing starts. For the engine methods, the time of update t is the wall-clock time (`std::chrono::steady_clock`) of processing its block of atomic edits: the store updates, the work of the method for each edit, the recomputation of the static baselines, and any rebuild, which is charged to the update that triggers it. Verification and output are excluded. Building the structure for K_0 is timed separately and reported as build time. For gudhi, the time of an update covers the simplex-tree edits, `compute_persistence` and `betti_numbers`, again without parsing.

All runs are single-threaded processes. Each configuration ran with 3 seeds, which change the generated workload; W5a is determined by L alone, so its three runs are repetitions of one instance. Every run processed $T = 2{,}000$ updates, except gudhi on instances with more than 300,000 simplices, which processed the first $T' = 200$ (hollow markers in the

Table 2: Experiments. Parameters not listed are the generator defaults of Section 6.3. "as E1": full-id, full-sorted, full-morse, gudhi, drb-id, drb-id-rb, mmhm, mmhm-norb; "DRB methods": the last four. gudhi processes all updates on instances with at most 300,000 simplices and the first $T' = 200$ updates on larger ones.

	Workload	Sweep	Methods
E0	samples; all runs of E1–E8	—	all exact methods
E1	W2, $s = 8$	$N \in \{7, 10, 15, 22, 33\}$	as E1
E2	W2, $N = 22$	$s \in \{1, 2, 4, 8, 16, 32, 64\}$	DRB methods, full-id, full-morse
E3	W3, $k = 2$	$N \in \{14, 20, 27\}$	as E1
E4	W4	square, $D = 2$: $n \in \{1000, 4000, 16000\}$; square and cube, $D = 3$: $n \in \{1000, 4000\}$	as E1
E5	W1	sphere $L \in \{4, 5, 6, 7\}$; torus $n = m \in \{64, 128, 256, 500\}$	as E1, and euler
E6	W5a, W5b	(a) $L = 10^3, 3 \cdot 10^3, 10^4, \ldots, 10^6$; (b) levels $L = 3, \ldots, 8$	DRB methods, full-id, gudhi; euler on (b)
E7	W2 $N = 22$, W3 $N = 20$, W4 square $n = 16000$, $D = 2$	$\alpha \in \{0.25, 0.5, 1, 2, 4, \infty\} \times m \in \{\infty, 256, 64\}$	mmhm, drb-id-rb
E8	W2 $N = 22$, W3 $N = 20$, W4 as in E7, W1 torus 256×256; for drb-rand one size per family (Table 13)	runs of E1, E3, E4, E5	DRB methods; drb-rand

figures, $'$ in the tables). For each instance and method we pool the per-update latencies of the three runs and report their median, interquartile range and 95th and 99th percentiles (p95, p99), and the *amortised latency*: the total processing time of all updates, rebuilds included, divided by their number (the initial build is not included; we report it separately). A *speedup* is a ratio of amortised latencies over the same updates; for gudhi on a prefix, both methods are taken over that prefix. Slopes are least-squares fits of log latency against log size. All 1,344 configured runs finished (0 were not run); they took 6.5 hours of single-threaded processing in total.

Drift of the static baseline. Because the insertion order of full-id depends on the history, its cost is not stationary. Its median latency over the last 200 updates exceeds that over the first 200 by more than 25 % on 10 instances: every W3 instance (up to a factor 2.86), every W4 instance with $n < 16000$, and the 64×64 torus (Table 7). full-sorted, whose order cannot drift, drifts by at most 1.09 on W1 and W2, by 1.01 to 1.23 on W4 (full-id: 1.15 to 1.64), and by 1.44 to 1.84 on W3, where part of the drift therefore comes from the changing complex rather than from the order. The DRB methods drift as well; on W3 mmhm drifts less than full-id. Where the drift matters we give speedups over matched windows of the first and the last 200 updates and against full-sorted.

Noise. Five identical runs of mmhm and full-id on W2 with $N = 22$ gave coefficients of variation (CV) of the amortised latency of 8.4 and 4.5 % and of the p99 of 12.0 and 14.0 %. On the W1 torus 256×256, W3 with $N = 20$ and W5a with $L = 10^5$ (five runs each of mmhm, drb-id and full-id) the amortised CV was 0.6 to 6.7 % and the p99 CV 1.1 to 10.5 %; on microsecond-scale updates (W5a, $L = 1000$) the CV reaches 28.9 % (amortised) and 39.6 % (p99). At most two timed runs executed at once. Concurrent load changed the median within the run-to-run spread at $N = 22$, but at 955,059 simplices three busy processes raised it by $+14.2$ % (full-id) and $+2.5$ % (mmhm); untimed verification jobs overlapped 46 timed runs early in the sweep (`experiments/ENVIRONMENT.md`). We call a difference between two methods *within noise* when their per-seed ranges overlap (`experiments/results/summary_pairs.csv`). Each range has three points, and on W1–W4 the seeds are different instances, so this is a weak criterion, not a test.

Counters. Besides time, every run records per update the number of column additions, the sum of the footprints $|\operatorname{occ}(\sigma)|$ over the deleted simplices, and $\operatorname{nnz}(R) + \operatorname{nnz}(V)$; in an update with a rebuild, the additions of the rebuild and of the edits are summed. These counts are proxies for the cost terms of Proposition 3: they count terms, not the sizes of the chains involved (Section 5.1), so they need not move with time. They do not depend on the machine.

6.5 Correctness protocol

The reported Betti numbers are compared with up to three references: (1) the engine's full-id, a static recomputation that shares the parser and the complex store with the dynamic methods but none of their reduction code, and is therefore *engine-internal*; (2) the closed-form Betti numbers of W1, W2, W3 and W5; and (3) GUDHI, an independent library

Table 3: Correctness (E0): per exact method, the runs, the Betti vectors they reported, the states compared with the full-id output of the same instance, with the closed-form values and with GUDHI, and the mismatches (drb-rand ran on four instances only).

method	runs	states	vs. full-id	vs. analytic	vs. GUDHI	mismatches
full-id	126	252,126	(ref.)	210,105	203,526	0
full-sorted	69	138,069	138,069	96,048	111,069	0
full-morse	87	174,087	174,087	132,066	147,087	0
gudhi	126	203,526	203,526	161,505	(ref.)	0
drb-id	126	252,126	252,126	210,105	203,526	0
drb-id-rb	288	576,288	576,288	426,213	527,688	0
drb-rand	12	24,012	24,012	18,009	24,012	0
mmhm-norb	126	252,126	252,126	210,105	203,526	0
mmhm	288	576,288	576,288	426,213	527,688	0
euler	42	84,042	84,042	84,042	57,042	0
all exact	1,290	2,532,690	2,280,564	1,974,411	2,005,164	0

Table 4: Coverage of the correctness checks (E0): Betti vectors of exact runs compared with at least one, at least two, and all three references (the engine's full-id, the closed form, GUDHI), and with at least one external reference (closed form or GUDHI).

workloads	vectors	≥ 1 ref.	≥ 2 ref.	all 3	≥ 1 external
W5 witnesses	482,652	482,652 (100.0%)	461,052 (95.5%)	250,974 (52.0%)	482,652 (100.0%)
W4 flag complexes	558,279	558,279 (100.0%)	474,237 (84.9%)	0 (0.0%)	516,258 (92.5%)
W1 surfaces	422,019	422,019 (100.0%)	405,819 (96.2%)	228,771 (54.2%)	422,019 (100.0%)
W3 cavities	360,783	360,783 (100.0%)	355,383 (98.5%)	297,765 (82.5%)	360,783 (100.0%)
W2 refine/coarsen	708,957	708,957 (100.0%)	703,557 (99.2%)	549,891 (77.6%)	708,957 (100.0%)
all exact runs	2,532,690	2,532,690 (100.0%)	2,400,048 (94.8%)	1,327,401 (52.4%)	2,490,669 (98.3%)
Python reference (`reference/`): 8 committed samples only, 3 methods, 2,514 vectors, 0 mismatches					

in another language, which replays the same sequence. The two external references are (2) and (3). A pure-Python implementation of the dynamic reduced basis and of the Morse order, written from the specification and sharing no code with the engine, ran only on the 8 committed samples.

Every exact run records the Betti numbers it reports after every update. full-id processes all T updates on every instance, and every other exact run is compared state by state with the full-id output of the same instance. The runs are also compared with the closed-form Betti numbers where these are known (W1, W2, W3, W5), and with the GUDHI oracle on all updates of instances with at most 300,000 simplices and on the first T' updates of larger ones. In addition, every exact engine run compares itself with an internal recomputation after K_0, after every 50th update and after the last one, outside the timed region. On the 8 committed samples, all exact engine configurations (including aggressive rebuild settings), the reference implementation, GUDHI and the closed-form values are cross-checked on every state. A mismatch of an exact method stops the whole pipeline. Table 4 gives the resulting coverage.

Below the level of outputs, the engine's randomised tests check (I1)–(I3) and compare every exact method with a brute-force rank computation on random edit sequences, and the reference implementation checks (I1)–(I3) after every atomic edit, Lemma 2 and Corollaries 2.1 and 2.2 on random complexes and Proposition 4 on the W5 families.

6.6 E0: Are the exact methods exact?

Yes, in every state that was checked, and every state was checked against at least one reference.

The 1,290 runs of exact methods reported 2,532,690 Betti vectors (Table 3). Every one was compared with at least one reference, 94.8 % with at least two, 98.3 % with at least one external reference (closed form or GUDHI) and 52.4 % with all three (Table 4); no comparison found a mismatch (0). The 42,021 vectors checked only against the engine-internal full-id are those of the gudhi runs on W4, which has no closed form. In addition, the exact engine runs made 47,724 internal verifications against a recomputation, and on the 8 committed samples the Python reference checked 2,514 vectors of 3 methods (0 mismatches).

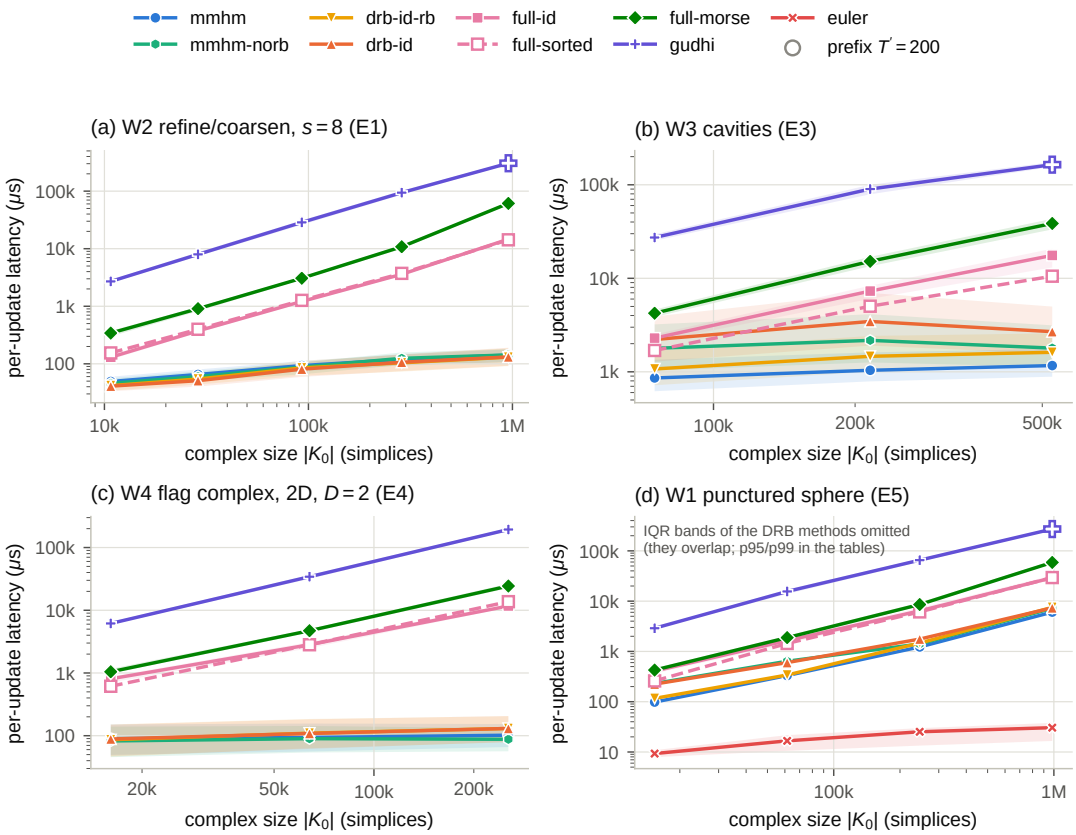


Figure 2: Median per-update latency against $|K_0|$ at a fixed edit size (log–log; band: interquartile range; three runs pooled). (a) W2 with $s = 8$ (E1); (b) W3 (E3); (c) W4 in the square with $D = 2$ (E4); (d) W1 punctured sphere (E5), without the bands of the DRB methods, which overlap. Hollow markers: gudhi on the prefix T'.

6.7 E1: Does the per-update cost depend on the size of the complex at a fixed edit size?

Only weakly for the dynamic reduced basis, and linearly for static recomputation. On W2 with $s = 8$, the median per-update latency of the four DRB methods grows with $|K_0|$ with slope 0.24 to 0.27 over five sizes, against 1.04 for full-id, 1.06 for gudhi and 1.14 for full-morse (Figure 2(a)).

At the largest size, $|K_0| = 955{,}059$, the four DRB variants are 90.2 to 91.5 times faster than full-id, and their differences are within noise: no rebuild fired, and the insertion order is as fast as the Morse order. On W2 the gain is that of the dynamic reduced basis, not of the Morse order or of rebuilds. mmhm takes 165 µs per update amortised against 14,900 µs for full-id, and its p99 is 333 against 21,600 µs (Tables 5 and 11). Over the same first T' updates, mmhm is 948 times faster than gudhi. The DRB pays more at the start of a run: over the first 200 updates mmhm is 46.0 times faster than full-id, over the last 200 103 times, although its median hardly changes (138 and 143 µs); a few early updates are expensive. full-sorted, which does not drift on W2 and is within noise of full-id at this size, gives the same picture: mmhm is 88.6 times faster over the whole run, 44.2 and 100 times over the two windows. Building the structure for K_0, including the insertion into the store, took 719 ms for mmhm and 667 ms for full-id, and $\mathrm{nnz}(R) + \mathrm{nnz}(V)$, a proxy for the memory of the DRB, reached at most 3.80 entries per simplex.

One run is an outlier: at $N = 22$, one of the three mmhm-norb runs was slow in its first 400 updates only (median 581 µs, against 118 and 125 µs for the other two). A rerun produced identical Betti numbers, additions, nnz and footprints and normal timings (amortised 114 against 252 µs); the tables keep the original run.

Static recomputation in the Morse order. full-morse is slower than full-id and full-sorted on every instance on which they ran, than full-id by factors from 1.09 (W1 sphere, $L = 4$) to 4.22 (W2, $N = 33$; Table 9). There is little for the collapse to save: on W2 the insertion order needs only 192 additions per recomputation, and on the largest W4 instance full-morse performs more additions than full-id (429,000 against 398,000). For one-shot static recomputation the collapse does not pay for itself on our workloads.

Table 5: Summary at the largest size of each family of E1, E3, E4 and E5: $|K_0|$, the number T of timed updates per run, the median, p95 and amortised per-update latency in µs (three runs pooled), and the speedup over full-id (ratio of amortised latencies over all T updates). gudhi entries with $T = 200$ cover only the first T' updates, and on W3 full-id drifts (Table 7); the text gives matched comparisons. Every instance is listed in Tables 9 and 11.

instance		$\lvert K_0\rvert$	method	T	median	p95	amortized	× full-id
W1 sphere $L = 7$	(E5)	983,042	full-id	2,000	29,100	44,700	30,700	–
			full-sorted	2,000	29,400	44,800	31,100	0.986
			full-morse	2,000	58,900	82,000	61,500	0.499
			gudhi	200	272,000	317,000	272,000	0.113
			drb-id	2,000	7,340	218,000	37,100	0.827
			drb-id-rb	2,000	7,520	231,000	39,200	0.782
			mmhm-norb	2,000	7,160	82,000	15,500	1.98
			mmhm	2,000	6,050	86,100	16,600	1.85
			euler	2,000	30.5	49.7	29.5	1,040
W1 torus 500×500	(E5)	1,500,000	full-id	2,000	34,700	65,800	38,200	–
			full-sorted	2,000	36,700	66,600	40,000	0.955
			full-morse	2,000	94,600	125,000	98,200	0.389
			gudhi	200	421,000	503,000	427,000	0.0896
			drb-id	2,000	10,900	246,000	45,100	0.848
			drb-id-rb	2,000	13,000	335,000	57,600	0.663
			mmhm-norb	2,000	11,700	184,000	33,400	1.14
			mmhm	2,000	11,500	183,000	34,000	1.13
			euler	2,000	32.1	50.9	30.1	1,270
W2 $N = 33$, $s = 8$	(E1)	955,059	full-id	2,000	14,600	18,300	14,900	–
			full-sorted	2,000	14,300	19,000	14,600	1.02
			full-morse	2,000	61,200	80,400	62,800	0.237
			gudhi	200	305,000	354,000	307,000	0.0485
			drb-id	2,000	132	293	165	90.3
			drb-id-rb	2,000	131	285	163	91.5
			mmhm-norb	2,000	143	244	163	91.4
			mmhm	2,000	142	243	165	90.2
W3 $N = 27$	(E3)	525,043	full-id	2,000	17,600	25,500	17,100	–
			full-sorted	2,000	10,500	15,200	10,800	1.58
			full-morse	2,000	38,600	51,400	39,200	0.437
			gudhi	200	164,000	207,000	168,000	0.102
			drb-id	2,000	2,690	14,100	4,600	3.72
			drb-id-rb	2,000	1,620	6,460	2,500	6.85
			mmhm-norb	2,000	1,790	8,760	2,920	5.86
			mmhm	2,000	1,170	4,160	1,810	9.46
W4 2D $n = 16$k, $D = 2$	(E4)	254,196	full-id	2,000	11,700	15,300	12,000	–
			full-sorted	2,000	13,700	20,600	14,400	0.830
			full-morse	2,000	24,300	33,400	25,100	0.477
			gudhi	2,000	194,000	232,000	196,000	0.0611
			drb-id	2,000	131	393	169	71.0
			drb-id-rb	2,000	130	391	168	71.1
			mmhm-norb	2,000	87.7	236	110	109
			mmhm	2,000	102	265	127	94.5
W4 3D $n = 4$k, $D = 3$	(E4)	85,205	full-id	2,000	3,100	4,130	3,160	–
			full-sorted	2,000	3,220	4,880	3,440	0.921
			full-morse	2,000	6,150	8,150	6,290	0.503
			gudhi	2,000	50,400	62,500	51,300	0.0617
			drb-id	2,000	154	700	236	13.4
			drb-id-rb	2,000	161	740	245	12.9
			mmhm-norb	2,000	132	534	192	16.5
			mmhm	2,000	139	565	201	15.7

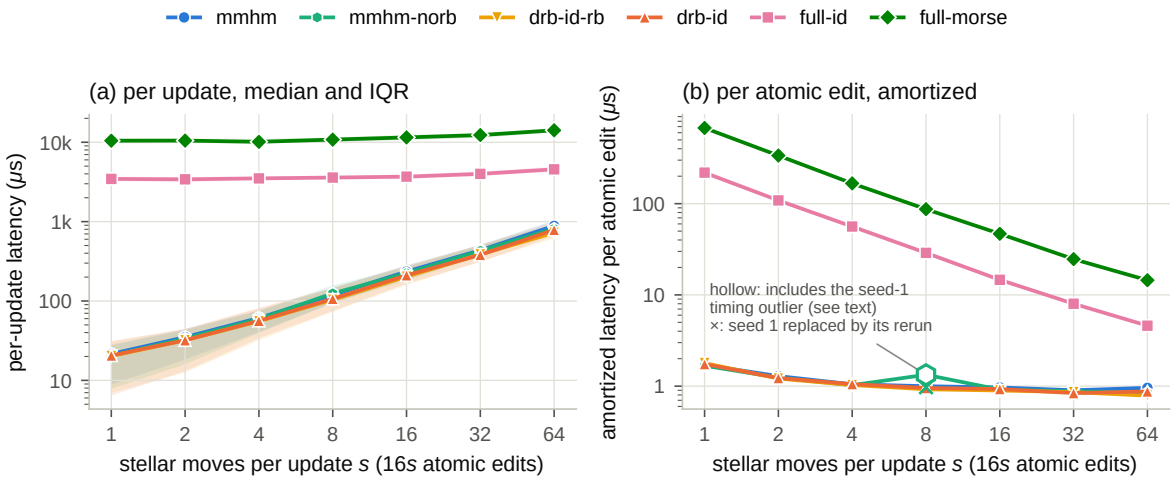


Figure 3: Cost against the number s of stellar moves per update (E2; W2, $N = 22$, $16s$ atomic edits per update; log–log). (a) Median per-update latency with interquartile range; (b) amortised latency per atomic edit. The hollow mmhm-norb point at $s = 8$ includes the outlier run of E1; $\times$ marks the same point with that run replaced by its rerun.

6.8 E2: How does the cost scale with the edit size?

Linearly for the DRB methods. On W2 with $N = 22$ ($|K_0|$ between 285,805 and 292,861), the median per-update latency of the DRB methods grows with the number s of stellar moves with slope 0.88 to 0.90, whereas full-id and full-morse barely depend on s (slopes 0.06 and 0.07; Figure 3(a)). The amortised cost per atomic edit of the DRB methods therefore stays in a narrow band, while that of recomputation falls roughly as $1/s$ (Figure 3(b)). There is no crossover in the measured range: at $s = 64$, that is 1024 atomic edits per update, the DRB methods are still 4.80 to 5.85 times faster than full-id.

6.9 E3: What does an update cost when the three-dimensional topology changes?

mmhm has the lowest amortised latency at every size of W3, beyond noise, but the DRB is not uniformly better than recomputation. We take full-sorted as the reference here: it is faster than full-id at every size and drifts less on the larger instances (Section 6.4). Against it, the DRB methods without rebuilds are slower at $N = 14$ and have a higher p99 at $N \leq 20$; the rebuilding ones are faster at every size.

At $N = 14$ ($|K_0| = 74{,}957$), mmhm takes 1,230 µs amortised, full-sorted 1,790 and full-id 2,320 µs; drb-id (3,180 µs) and mmhm-norb (2,600 µs) are slower than full-sorted, and at $N = 20$ drb-id still is (speedup 0.794). The p99 of full-sorted, 3,550 µs, is below that of every DRB method at $N = 14$ except mmhm (5,110 µs, within noise). At $N = 27$ ($|K_0| = 525{,}043$) every DRB method is faster than both static baselines on average. mmhm is 5.99 times faster than full-sorted over the whole run (7.45 and 8.44 over the first and the last 200 updates) and 9.46 times faster than full-id: full-id is 0.957 times as fast as full-sorted over the first 200 updates and 0.564 times as fast over the last 200, as its order drifts (Table 7). Over the same first T' updates mmhm is 160 times faster than gudhi. At $N = 27$ the p99 of mmhm and drb-id-rb is below that of full-sorted, that of mmhm-norb within noise of it, and that of drb-id above it. At $N = 20$ and $N = 27$ the amortised latencies are ordered mmhm, drb-id-rb, mmhm-norb, drb-id, all beyond noise (at $N = 14$ the last two are within noise): rebuilding matters more than the order, and the Morse order helps both with and without rebuilds.

The basis grows over a W3 run (Figure 4(b); it also grows on W1, E5). At $N = 20$, without rebuilds, $\mathrm{nnz}(R) + \mathrm{nnz}(V)$ grows from 812,580 to 4,505,833 (drb-id) and to 3,734,680 (mmhm-norb), and the latency grows with it. The growth trigger rebuilds mmhm 6.0 times and drb-id-rb 10 times per run (means over the runs), and the runs end at 1,483,381 and 1,667,205.

6.10 E4: How do the methods behave on non-manifold complexes in which every Betti number varies?

The DRB methods are faster than both static baselines on every W4 instance, their latency hardly depends on $|K_0|$, and on the larger instances the Morse order helps. In the square with $D = 2$ (Figure 2(c)), their median latency grows with slope 0.02 to 0.15, against 0.97 for full-id (three sizes, so the slopes are indicative only). Against full-sorted, our reference here because it drifts less than full-id on every W4 instance (Table 7), their speedup is 3.25 to 6.09 at $n = 1000$ and 13.3 to 132 at $n \geq 4000$; against full-id it is 3.72 to 7.64 and 12.2 to 109. At $n = 16000$ full-sorted is slower than

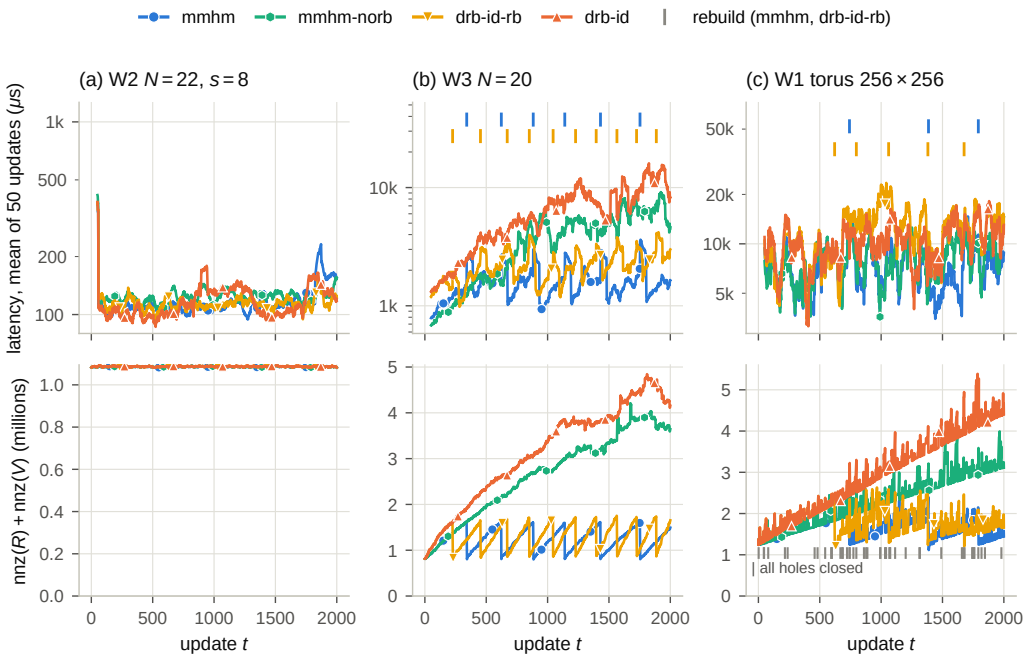


Figure 4: Behaviour over time of the four DRB methods (seed 0). Top: per-update latency, mean of a sliding window of 50 updates, with the rebuilds of mmhm and drb-id-rb marked. Bottom: $\mathrm{nnz}(R) + \mathrm{nnz}(V)$ after each update. (a) W2 with $N = 22$, $s = 8$ (E1); (b) W3 with $N = 20$ (E3); (c) W1 torus 256×256 (E5), with the updates after which no hole is open marked.

full-id (14,400 against 12,000 µs amortised), and mmhm is 114 times faster than it over the whole run and 81.2 and 120 times over the first and the last 200 updates (against full-id: 94.5, 64.8 and 103). The p99 of the DRB methods is below that of both baselines on every W4 instance except the two with $n = 1000$ and $D = 3$, where the difference is within noise (Table 11).

mmhm-norb is faster than drb-id beyond noise on the square instances with $n \geq 4000$; on the others the difference is within noise. At $n = 16000$ the amortised latencies are 110 and 169 µs, the p99 latencies 353 and 644 µs, and the p99 of the column additions per update 1,130 and 2,130 (E8). Under the default policy, rebuilds fired in only one W4 run of each rebuilding method (in the cube, $n = 1000$, $D = 3$). Without rebuilds, mmhm does the same additions as mmhm-norb (Table 10) but is slower at $n = 16000$ (127 against 110 µs, beyond noise). We attribute this, without having profiled it, to the index of cells by vertex tuple that mmhm maintains for a possible Morse rebuild. It is consistent with E7, where mmhm with both triggers off, which never rebuilds but keeps the index, has a median of 98.7 µs against 87.7 µs for mmhm-norb.

6.11 E5: On surfaces, what does linear algebra cost compared with combinatorial invariants?

On average the DRB is at best about twice as fast as recomputation, sometimes slower, and it has the worse tail; the combinatorial counters are far cheaper than both. On no W1 instance is a DRB method more than 2.06 times faster than full-id, and on several smaller instances mmhm-norb is within noise of it. Against full-sorted, drb-id is slower on every W1 instance (within noise on the 256×256 torus), mmhm-norb is slower on the four smallest, and no DRB method is more than 2.00 times faster. On the largest sphere and torus the insertion-order variants are slower than full-id and the Morse-ordered ones faster, all beyond noise. Every DRB variant has a higher p99 than full-id on every W1 instance.

On the sphere with $L = 7$ ($|K_0| = 983{,}042$), full-id takes 30,700 µs amortised, drb-id 37,100 and drb-id-rb 39,200 µs; mmhm-norb and mmhm are 1.98 and 1.85 times faster than full-id. On the torus with $n = m = 500$ ($|K_0| = 1{,}500{,}000$) the speedups are 0.848 (drb-id), 0.663 (drb-id-rb), 1.13 (mmhm) and 1.14 (mmhm-norb). The median tells a different story from the average: at $L = 7$ the median of mmhm is 4.82 times below that of full-id (6,050 against 29,100 µs), while its p99 is 151,000 against 52,200 µs; the average is set by a heavy tail (Tables 5 and 11). euler needs 10.5 to 30.1 µs per update, including the edits of the complex store, 1,040 times less than full-id on the largest sphere.

The median of the DRB methods grows with $|K_0|$ almost like that of full-id (slopes 0.80 to 1.01 on the sphere, against 1.02; Figure 2(d)), so a typical W1 update presumably works on chains whose size grows with the complex (chain sizes were not logged). The closed surface, on which Proposition 4 forces $\Omega(n_2)$ work, reappears after only 38.0 of the

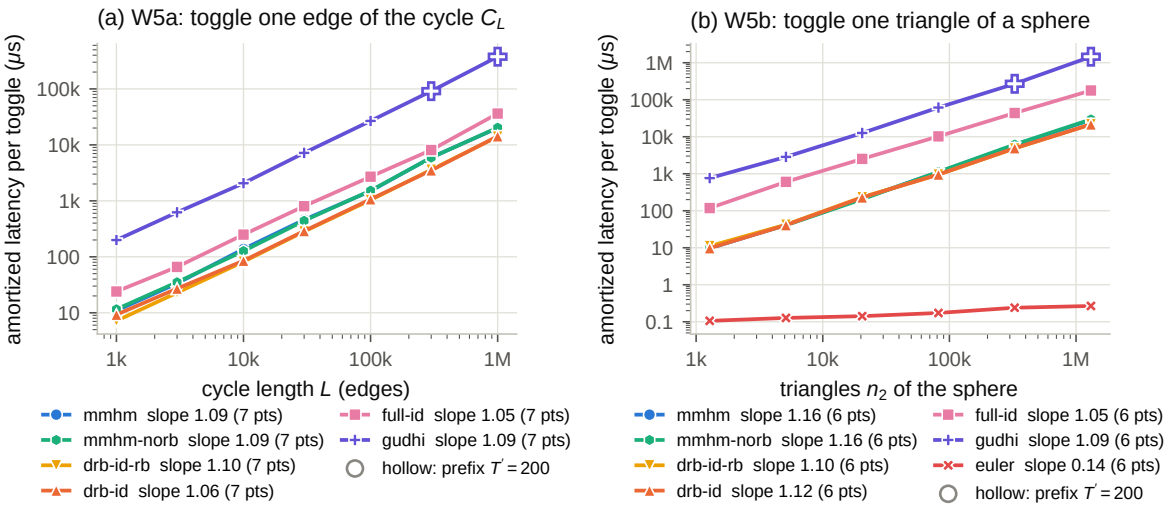


Figure 5: Amortised latency per toggle on the non-locality witnesses (E6; log–log, three runs pooled; the legend gives each least-squares slope). (a) W5a against the cycle length L; (b) W5b against the number n_2 of triangles. Hollow markers: gudhi on the prefix T'.

2,000 updates of a run on the largest sphere (mean over the runs), too rarely to set the median, and on that sphere the median of mmhm-norb is nearly the same over the first and the last 200 updates (6,970 and 7,650 µs). We conjecture that typical updates meet dense cycle slots, such as those left by the Case A deletion that opens a hole in the closed surface (Proposition 4(a)); we have not measured chain sizes. Between rebuilds the basis also becomes denser (Figure 4(c)): on the largest torus $\mathrm{nnz}(R) + \mathrm{nnz}(V)$ grows over a run by a factor 2.46 for drb-id and 1.84 for mmhm-norb. This is Remark 5.1 in practice: on connected surfaces with known connectivity, the counters are correct and much cheaper, and the algebra is not needed.

6.12 E6: Where does locality fail?

Where Proposition 4 says it does. On both witness families, the cost of every DRB method per toggle grows linearly with the size of the toggled cycle. The DRB methods stay faster than full-id on average, but their p99 is higher on the cycle graph, and for the Morse-ordered variants also on the largest sphere; only euler (on the sphere) is nearly independent of the size.

The amortised latency per toggle grows with slope 1.06 to 1.10 on W5a (L from 10^3 to 10^6) and 1.10 to 1.16 on W5b (levels 3 to 8), against 1.05 and 1.05 for full-id (Figure 5). At $L = 10^6$, drb-id takes 14,200 µs, mmhm 20,100 µs and full-id 36,300 µs; on the largest sphere ($|K_0| = 3{,}932{,}162$) the DRB methods take 21,500 to 29,400 µs and full-id 178,000 µs. On W5a the p99 of every DRB method exceeds that of full-id at every L, beyond noise except for mmhm at $L = 1000$ (Table 11); at $L = 10^6$ it is 101,000 µs for mmhm against 48,100 µs. On W5b at level 8 the p99 of mmhm and mmhm-norb is 1.13 and 1.15 times that of full-id, that of the insertion-order variants 0.849 and 0.859 times. The Morse order does not help here: at $L = 10^6$, drb-id is faster than mmhm beyond noise, and the reference implementation likewise counts more elementary operations per toggle under the Morse order than under insertion order on these families (`reference/README.md`); we have not identified why. euler takes 0.106 to 0.267 µs per toggle on W5b (slope 0.14).

6.13 E7: How sensitive is mmhm to the rebuild policy?

It depends on the workload, and no setting is best everywhere. On W3, where the basis grows, rebuilding by the growth trigger pays off and the Morse order at a rebuild helps. On W2 and W4 the trigger hardly ever fires and periodic rebuilds only add cost (Figure 6). On the surfaces of W1, where the basis also grows, rebuilds help on the smaller instances and not on the largest (Table 8).

W2 and W4. With the growth trigger alone, no rebuild fired on W2 at any α, and on W4 one mmhm run of three rebuilt once, at $\alpha = 0.25$ (0.33 rebuilds per run). Periodic rebuilds raised the amortised latency of mmhm on W2 from 123 µs ($\alpha = 1$, no period) to 265 µs ($m = 256$) and 654 µs ($m = 64$). A Morse rebuild, which includes a collapse, costs more than an insertion-order one: drb-id-rb took 432 µs at $m = 64$. mmhm is faster than drb-id-rb at only 2 of the 18 settings on W2 and at 8 on W4.

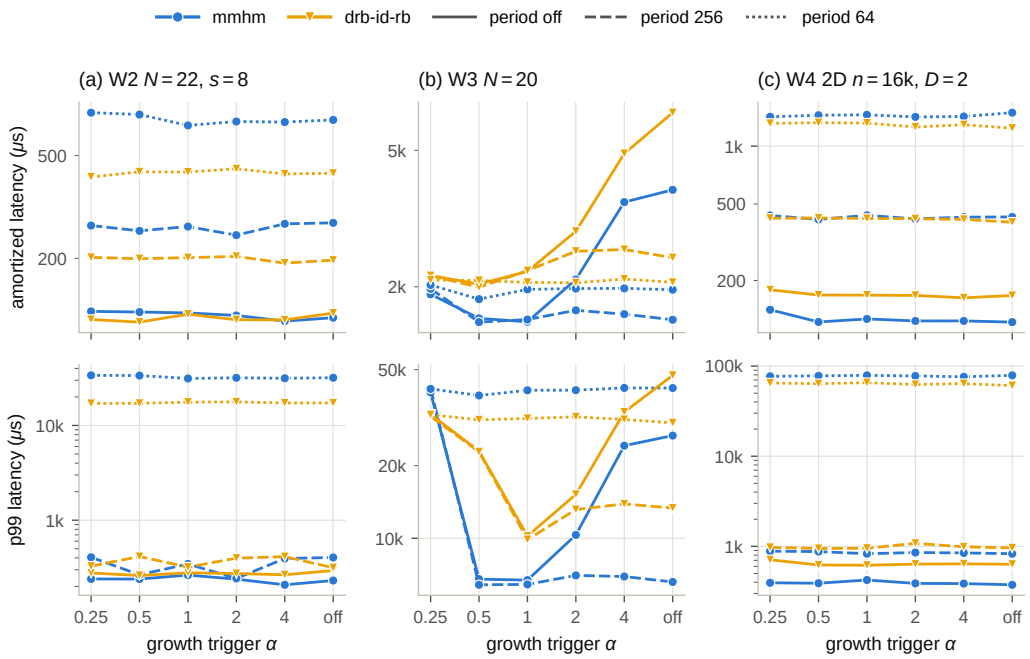


Figure 6: Rebuild policy (E7): amortised (top) and p99 (bottom) per-update latency of mmhm and drb-id-rb against the growth threshold α, one line per period m; three runs pooled. (a) W2 with $N = 22$, $s = 8$; (b) W3 with $N = 20$; (c) W4 in the square with $n = 16000$, $D = 2$. With $\alpha = m = \infty$ ("off") the methods never rebuild. E7 has no W1 instance; W1 rebuild data are in Table 8.

W3. At $\alpha = 1$ without a period, mmhm took 1,580 µs amortised against 3,830 µs without rebuilds, and drb-id-rb 2,220 against 6,440 µs; the p99 of mmhm fell from 26,600 to 6,710 µs. Too small a threshold hurts again: at $\alpha = 0.25$ mmhm rebuilt 35 times per run and its p99 was 40,300 µs. On W3 mmhm is faster than drb-id-rb at 18 of the 18 settings. Under the default policy, the updates after the first rebuild need fewer additions than the same updates of the never-rebuilding twin (ratio 0.85 for mmhm, 0.78 for drb-id-rb at $N = 20$) and much less time (0.33 and 0.29), on a much sparser basis (Figure 4(b)).

W1. Under the default policy rebuilds fire on every W1 instance (Table 8). Against its never-rebuilding twin, mmhm is faster beyond noise on the sphere with $L \leq 5$ and the tori with $n \leq 128$ (for instance 837 against 1,910 µs at $L = 5$), within noise on the sphere with $L = 7$ and the tori with $n \geq 256$, and slower beyond noise on the sphere with $L = 6$. drb-id-rb is faster than drb-id on the sphere with $L \leq 5$ and the tori with $n \leq 128$, within noise on the spheres with $L \geq 6$, and slower beyond noise on the tori with $n \geq 256$. In the 100 updates after an mmhm rebuild, mmhm performs more additions than mmhm-norb on the same updates on every W1 instance except the sphere with $L = 5$, by factors up to 2.52. Over all updates after the first rebuild, the ratio of additions is below one on the large spheres (0.95 and 0.92) and above one on the largest torus (1.21), while the ratio of time is 1.10 and 1.09 on those spheres. The additions and the time disagree here, as the counters are only proxies (Section 5.1); we have not identified the mechanism.

6.14 E8: What does the Morse order buy?

At build time nothing against the insertion orders of our generators, although both structured orders are far better than a random one. Between rebuilds it depends on the workload: the Morse order makes the DRB faster on surfaces, on cavities and on the larger flag complexes, makes no difference on W2, and makes it slower on the non-locality witnesses.

At build time. Lemma 2 and Corollaries 2.1 and 2.2 describe the build, so we compare the builds of drb-id and mmhm-norb (Table 6). The insertion orders written by our generators already need almost no additions on W2 (192 at $N = 33$, against 0 in the Morse order) and none on W3; on W1 and W5 both orders need the same number within one percent (for instance 327,806 and 327,679 on the sphere with $L = 7$). On W4 the Morse order needs more additions, by factors 1.04 to 1.15, and yields a slightly smaller basis ($\mathrm{nnz}(R) + \mathrm{nnz}(V)$ ratio 0.963 to 0.995). The lemma and its corollaries thus say where the work of a build is done, not that there is less of it than in a good insertion order.

Against a random order. The insertion orders of our generators are themselves structured. Built in a uniformly random order (drb-rand, one instance per family, Table 13), the basis has 1.84 to 2.29 times as many entries after INIT as

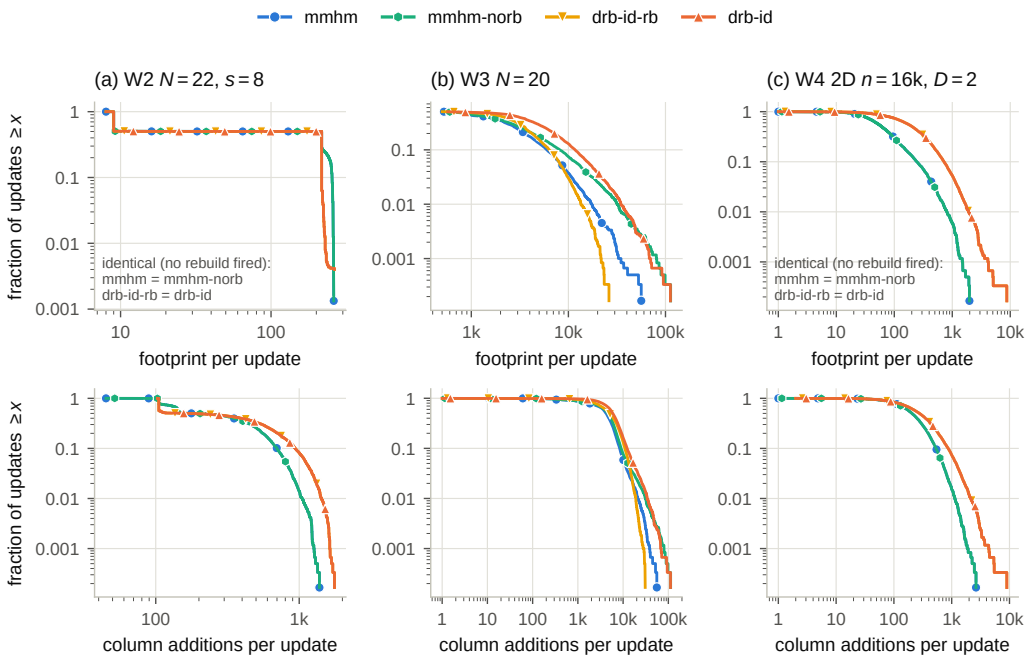


Figure 7: Counters between rebuilds (E8): complementary cumulative distributions of the footprint per update (top; the sum of $|\operatorname{occ}(\sigma)|$ over its deletions) and of the column additions per update (bottom), over the updates without a rebuild (three runs pooled; log–log). (a) W2 with $N = 22$, $s = 8$; (b) W3 with $N = 20$; (c) W4 in the square with $n = 16000$, $D = 2$. On W2 and W4 no rebuild fired, so mmhm coincides with mmhm-norb and drb-id-rb with drb-id. The W1 torus of Table 10 is not shown.

in either structured order, and the build needs more additions on every instance (on W2 72,361, against 192 in insertion and 0 in Morse order). On W1, W2 and W3 the updates are slower beyond noise, by factors 1.50 to 3.94 against either structured order (on W2 348 against 88.4 and 91.1 µs); there the choice between the two structured orders matters less than using one. On W4 drb-rand is within noise of drb-id and 1.48 times slower than mmhm-norb.

Between rebuilds. Comparing the two methods that never rebuild, mmhm-norb against drb-id, the Morse order is faster beyond noise on the W1 spheres with $L \geq 5$ and on all W1 tori (on the sphere with $L = 7$, 15,500 against 37,100 µs), on W3 with $N \geq 20$ and on W4 in the square with $n \geq 4000$. It is within noise on W2 (except on the instance with the outlier run of E1) and on the smaller W3 and W4 instances, and slower beyond noise on W5a for every $L \geq 3000$ (at $L = 10^6$, 20,300 against 14,200 µs) and on W5b from level 6 on. Figure 7 and Table 10 show the counters on the non-rebuild updates of one instance each of W2, W3, W4 and W1; the footprint of an update is the sum of $|\operatorname{occ}(\sigma)|$ over its deleted simplices. Without rebuilds the Morse order gives lighter tails of the column additions on all four: on W4 ($n = 16000$) the p99 falls from 2,130 to 1,130 and the median footprint from 204 to 60.0. Not every quantile improves: on W2 the median additions rise from 156 to 166 and the p99 footprint from 228 to 259, and with rebuilds the insertion-order rebuild of drb-id-rb leaves lighter footprint tails than the Morse rebuild (p99 13,400 against 17,300 on W3), although mmhm has the lower amortised latency there (E7). Between rebuilds the Morse order is a heuristic (Remark 4.3), and these measurements are its whole justification.

7 Discussion and limitations

Two regimes. On W2, W4 and the larger W3 instances, the latency of the DRB methods depends little on the size of the complex. On W5, Proposition 4 forces a cost linear in the size of the toggled cycle, and the measurements follow it. On W1 the median cost of an update also grows with the complex, although typical updates create only small cycles; we have not measured why (E5). Where the cost grows with the complex, recomputation is also linear, so the DRB keeps at most a constant factor, and its tail latency can be worse (E5, E6). Five workload families do not settle which properties of a workload decide the regime.

7.1 Limitations

Coefficients and dimension. Every result of this paper is over $\mathbb{Z}_2$. Ranks over a field do not see torsion, which requires the Smith normal form over the integers [35], and Betti numbers over $\mathbb{Z}_2$ can differ from those over other fields: the real projective plane has Betti numbers $(1, 1, 1)$ over $\mathbb{Z}_2$ and $(1, 0, 0)$ over the rationals [24]. We have neither implemented nor tested other coefficients. The proofs of Sections 3 to 5 hold in every dimension, but the engine and all experiments are restricted to dimension at most three.

Worst-case cost and proxies. By Proposition 4 the cost of an update is bounded only in output-sensitive terms (Proposition 3), and what we log are proxies for those terms: counts of additions and footprints, not the sizes of the chains involved. The proxies and the time can disagree (E7). Where no spanning cycle is created the proxies stayed small: on the W2 and W4 instances of E8 the median number of column additions per update was at most 276. Nothing guarantees this for other workloads.

Rebuild policy and the Morse order. The growth threshold α and the period m are heuristics whose effect depends on the workload (E7), and a method that may rebuild in the Morse order also pays, we believe, for the index such a rebuild needs (E4). The results of Section 4 locate the work of a build in the critical columns; they do not make it smaller than in a good insertion order, and on our workloads it was not (E8). Between rebuilds the Morse order is a heuristic whose measured effect ranges from positive to negative, and for one-shot static recomputation the collapse did not pay for itself (E1).

Baselines. The insertion order of full-id depends on the history of edits, and its cost drifts upward during the W3 and the smaller W4 runs; the canonical order of full-sorted removes most of this drift on W4 but not on W3. Against it the gains of the DRB on W3 are smaller (mmhm at $N = 27$: 5.99 instead of 9.46) and the methods without rebuilds lose on the smallest instance. gudhi runs through its Python interface and computes persistence pairs; its times are a reference point for a widely used library, not the best possible static recomputation. We did not compare against implementations of zigzag persistence.

Measurements. All timings come from one virtual machine, single-threaded runs and one compiler setting. Run-to-run variation is a few percent on most instances we repeated and much larger on microsecond-scale updates (Section 6.4); we mark differences within the per-seed ranges as within noise. The counters do not depend on the machine. We did not measure memory; as a proxy, $\mathrm{nnz}(R) + \mathrm{nnz}(V)$ reached at most 3.80 entries per simplex on the largest W2 instance, 8.93 on the largest torus (drb-id) and 18.05 on the largest W3 instance (drb-id; mmhm: 7.56). W4 is truncated at dimension D, so it measures the algebra on non-manifold complexes, not the topology of point clouds.

7.2 When to use what

Surfaces with known connectivity: combinatorial invariants (Remark 5.1). On the largest W1 sphere, euler was 1,040 times faster than static recomputation and faster still than every DRB method.

Barcodes: zigzag persistence [7]. $\beta_k(K_t)$ is the number of k-dimensional intervals of the zigzag barcode that contain t, so zigzag algorithms [8, 13, 29, 31] answer our query as well, the incremental algorithm of Carlsson, de Silva and Morozov [8] after every update.

Connectivity only: fully dynamic connectivity, which maintains β_0 (and, for graphs, $\beta_1 = n_1 - n_0 + \beta_0$) in polylogarithmic amortised time per edge edit [25]; on W5a no method that stores cycle representatives can match this. For complexes in the 3-sphere built by insertions only, the algorithm of Delfinado and Edelsbrunner [11] needs near-linear total time.

Small complexes, or tail latency under global cycles: static recomputation with clearing, not in the Morse order. It stores no basis. In the canonical order it was faster on average than the DRB methods without rebuilds on the smallest W3 instance and than drb-id on almost every W1 instance; full-id had the lower p99 on every W1 instance and, beyond noise in all but one case, on every W5a instance. Within the edit sizes we measured ($s \leq 64$ on W2) it did not overtake the DRB methods on average; we did not test larger batches.

Edit-local updates on large general complexes: a dynamic reduced basis. On W2 every variant did equally well, so the gain there comes from the basis itself, not from the Morse order or rebuilds. Where the basis grows, as on W3, the growth-triggered rebuild in the Morse order (mmhm) was the fastest method at every size; on the larger W4 instances

and the large W1 surfaces the Morse order without rebuilds (mmhm-norb) was at least as fast; on the non-locality witnesses the insertion order was faster.

8 Related work

Static homology and persistence computation. Over a field, Betti numbers are ranks of boundary matrices, and the standard way to obtain them, together with the persistence barcode of a filtration, is column reduction in a fixed total order of the cells: each column receives additions from earlier columns until its lowest non-zero entry (its *low*) differs from that of every other non-zero column [17, 18, 39]. The worst case is cubic in the number of cells [17], and practical performance comes from a small set of devices. *Clearing* (the *twist*) processes dimensions from the top down and sets to zero every column whose cell is already known to be the low of a reduced column one dimension up [9]; *clear and compress* pairs columns locally in chunks and compresses the remainder before a global reduction [3]. *Apparent pairs* are decided by a purely local condition on the boundary matrix, need no column additions, and are both persistence pairs and gradient pairs of a discrete Morse function; Ripser combines them with clearing, cohomology and an implicit coboundary [1]. PHAT [4] and GUDHI [30] package these variants, and Otter et al. [36] benchmark the available software. All of this is static: an edited complex is a new input. Our rebuild step is exactly such a static reduction with clearing, modified to retain the change of basis V; our full-* baselines rerun it, without V, after every update.

Discrete Morse theory and coreductions. Forman's discrete Morse theory [20, 21] replaces a complex by a chain-homotopy-equivalent Morse complex on the critical cells of an acyclic matching. Algebraic Morse theory [26, 38] extends the construction to chain complexes of free modules and obtains the Morse complex by eliminating the matched pairs, whose incidences are invertible. Algorithmically, matchings are built by collapses or coreductions and used to shrink the input before any linear algebra: coreduction homology [33], extended to regular CW complexes [16], to maps and persistence [23, 34], and filtered Morse complexes that preserve persistent homology [32]. On 2-manifolds, matchings with the minimum number of critical cells can be constructed in linear time [27]; in general, Benedetti and Lutz [5] collapse a free face chosen at random and declare a cell critical only when no free face is left, and use the resulting distribution of critical-cell counts to measure how complicated a triangulation is. Our matching is a deterministic variant of their collapse strategy. We use it differently from the works above: we never form the Morse complex, whose boundary entries are sums over gradient paths, so that an edit to a *matched* cell can change the boundaries of critical cells far from the edit. Instead we use the matching only to choose the total order of the *unreduced* boundary matrix, so that every gradient pair is an apparent pair. This step is elementary and close to known results: the collapse removes each paired coface immediately before its face, so the two are adjacent in the reverse removal order, and a facet and a coface that are adjacent in a filtration order always form an apparent pair. In the other direction, the apparent pairs of any simplexwise filtration form a discrete gradient [1], and Bauer and Roll [2] relate the gradient pairs of a (generalised) discrete Morse function to the zero-persistence apparent pairs of a lexicographically refined filtration order. What we add is the consequence for the static reduction with clearing: all column additions happen in the columns of critical cells, and on critical rows and columns the reduction performs exactly the reduction of the Morse complex (Corollaries 2.1 and 2.2).

Updating persistence. Vineyards [10] maintain the decomposition $R = DV$ of a filtered boundary matrix, with V stored, under transpositions of adjacent cells at linear cost per transposition. Our dynamic reduced basis (DRB) is the same kind of object, a reduced matrix $R = BV$ with V invertible and tracked, maintained under a different set of moves. Zigzag persistence [7] handles both insertions and deletions. Carlsson, de Silva and Morozov [8] gave an incremental algorithm; Milosavljević, Morozov and Škraba [31] bounded the cost of n additions and deletions by matrix-multiplication time; Maria and Schreiber [29] use discrete Morse reductions as preprocessing, with cost depending mostly on the number of critical cells; and FastZigzag [13] converts a simplex-wise zigzag filtration into an ordinary filtration of a Δ-complex of the same length, so that standard persistence software applies. On graphs, zigzag persistence takes near-linear time using dynamic connectivity structures [12], and vineyard updates extend to zigzag filtrations, including expansions and contractions [14]. Dey, Fan and Wang [15] maintain $\mathbb{Z}_2$ simplex annotations under elementary inclusions and vertex collapses to compute persistence along simplicial maps. Closest to our deletion step are methods that warm-start from an existing $R = DV$ factorisation when a filtration changes. Luo and Nelson [28] extend the vineyard updates to cell insertions and deletions processed in a batch, with cost scaling with the number of elementary changes. Giunti and Lazovskis [22] update R, V and the representative cycles when a simplex and its cofaces are removed, compatibly with the twist (clearing) optimisation.

Incremental and dynamic Betti numbers. Delfinado and Edelsbrunner [11] compute the Betti numbers of a complex in the 3-sphere built by adding one simplex at a time, in $O(n\,\alpha(n))$ time. Each new simplex either creates a cycle or destroys one (the positive/negative dichotomy that is also our insertion step); in S^3 the test reduces to union–find on

the complex and on its complement, with β_1 following from the Euler characteristic. The algorithm is insertion-only and depends on the embedding. Fully dynamic β_0 under edge insertions and deletions is dynamic graph connectivity, solved in polylogarithmic amortised time per update [25]; for graphs embedded on a surface, Eppstein [19] maintains generators of the surface's fundamental group under edge insertions, deletions and their duals. These bounds rely on special structure (a graph, an ambient S^3, an embedding surface) and none of them covers β_1, β_2 and β_3 of an arbitrary complex of dimension at most three under insertions and deletions, which is our setting.

Positioning. Our dynamic reduced basis maintains $R_k = B_k V_k$ with V_k tracked under column insertion and column deletion, which suffices because rows appear and disappear only while they are zero. The deletion procedure is not new in substance: it is the rank-only specialisation of the deletion updates used for zigzag persistence and for warm-started $R = DV$ factorisations [8, 22, 28]; we give a self-contained proof. The difference lies in the output and in the freedom it buys. Zigzag persistence yields the barcode of the entire sequence of complexes, a strictly richer output, and an incremental zigzag algorithm [8] already answers our query. In zigzag, vineyard and warm-start methods, however, the cell order is the filtration and is fixed by the input. We report only the current Betti numbers, so the order is a computational device that we may choose (the Morse order) and replace at a rebuild; its measured effect is mixed (Section 6.14). For β_0 alone, and for insertion-only complexes in S^3, the specialised structures above have better worst-case bounds.

9 Conclusion

We have presented a dynamic reduced basis that maintains the $\mathbb{Z}_2$ Betti numbers of a simplicial complex exactly under insertions and deletions of simplices. Insertion is the standard reduction step; deletion corrects every basis chain that contains the deleted simplex, which is the rank-only form of known zigzag and warm-start updates. Exactness holds for every order of the cells. Discrete Morse theory enters through the order chosen at a build or rebuild: in the collapse order every gradient pair is an apparent pair, and the static reduction performs its column additions only in critical columns, where they are exactly the reduction of the Morse complex. The cost of an update is output-sensitive and cannot be bounded by the size of the update.

The experiments confirm the exactness and delimit where the method is useful. Where the edits touch only small cycles, the dynamic reduced basis is fast and nearly independent of the size of the complex: on the largest refinement instance all its variants are 90.2–91.5 times faster than static recomputation, a gain that owes nothing to the Morse order or to rebuilds. Where the cost grows with the complex, on surfaces and on the non-locality witnesses, the average gain is at most a constant factor or negative and the tail latency is often worse; on surfaces with known connectivity, combinatorial counters are the better tool. A structured order matters far more than which one: in a random order updates were up to 3.94 times slower, while the Morse order saved nothing at build time against the insertion orders of our generators and helped between rebuilds only on some workloads. Rebuilding paid off on cavities, did not trigger on refinement and flag complexes, and on surfaces helped the smaller instances but not the largest.

Open questions are maintaining the matching between rebuilds, so that the guarantees of Section 4 hold at all times; rebuild policies with provable guarantees; bounds on footprints and reduction chains for structured families of complexes and edits; other coefficients; and a comparison with implementations of zigzag persistence.

Code and data. Code, workloads, raw results and the scripts that regenerate every number and figure are available from the author. The file `experiments/README.md` describes how to regenerate them, and `experiments/results/WORKLOADS_MANIFEST.csv` records the generator command and the SHA-256 of every workload instance.

References

[1] Ulrich Bauer. Ripser: Efficient computation of Vietoris–Rips persistence barcodes. *Journal of Applied and Computational Topology*, 5(3):391–423, 2021. doi:10.1007/s41468-021-00071-5.

[2] Ulrich Bauer and Fabian Roll. Wrapping cycles in Delaunay complexes: Bridging persistent homology and discrete Morse theory. In Wolfgang Mulzer and Jeff M. Phillips, editors, *40th International Symposium on Computational Geometry (SoCG 2024)*, volume 293 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 15:1–15:16, 2024. doi:10.4230/LIPIcs.SoCG.2024.15.

[3] Ulrich Bauer, Michael Kerber, and Jan Reininghaus. Clear and compress: Computing persistent homology in chunks. In *Topological Methods in Data Analysis and Visualization III*, pages 103–117. Springer, 2014. doi:10.1007/978-3-319-04099-8_7.

[4] Ulrich Bauer, Michael Kerber, Jan Reininghaus, and Hubert Wagner. PHAT – persistent homology algorithms toolbox. *Journal of Symbolic Computation*, 78:76–90, 2017. doi:10.1016/j.jsc.2016.03.008.

[5] Bruno Benedetti and Frank H. Lutz. Random discrete Morse theory and a new library of triangulations. *Experimental Mathematics*, 23(1):66–94, 2014.

[6] Gunnar Carlsson. Topology and data. *Bulletin of the American Mathematical Society*, 46:255–308, 2009. doi:10.1090/S0273-0979-09-01249-X.

[7] Gunnar Carlsson and Vin de Silva. Zigzag persistence. *Foundations of Computational Mathematics*, 10:367–405, 2010. doi:10.1007/s10208-010-9066-0.

[8] Gunnar Carlsson, Vin de Silva, and Dmitriy Morozov. Zigzag persistent homology and real-valued functions. In *Proceedings of the Symposium on Computational Geometry (SoCG 2009)*, pages 247–256, 2009. doi:10.1145/1542362.1542408.

[9] Chao Chen and Michael Kerber. Persistent homology computation with a twist. In *Proceedings of the 27th European Workshop on Computational Geometry (EuroCG 2011)*, pages 197–200, Morschach, Switzerland, 2011.

[10] David Cohen-Steiner, Herbert Edelsbrunner, and Dmitriy Morozov. Vines and vineyards by updating persistence in linear time. In *Proceedings of the Symposium on Computational Geometry (SoCG 2006)*, pages 119–126, 2006. doi:10.1145/1137856.1137877.

[11] Cecil Jose A. Delfinado and Herbert Edelsbrunner. An incremental algorithm for Betti numbers of simplicial complexes on the 3-sphere. *Computer Aided Geometric Design*, 12(7):771–784, 1995. doi:10.1016/0167-8396(95)00016-Y.

[12] Tamal K. Dey and Tao Hou. Computing zigzag persistence on graphs in near-linear time. In *37th International Symposium on Computational Geometry (SoCG 2021)*, volume 189 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 30:1–30:15, 2021. doi:10.4230/LIPIcs.SoCG.2021.30.

[13] Tamal K. Dey and Tao Hou. Fast computation of zigzag persistence. In *European Symposium on Algorithms (ESA 2022)*, volume 244 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 43:1–43:15, 2022. doi:10.4230/LIPIcs.ESA.2022.43.

[14] Tamal K. Dey and Tao Hou. Computing zigzag vineyard efficiently including expansions and contractions. In *40th International Symposium on Computational Geometry (SoCG 2024)*, volume 293 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 49:1–49:15, 2024. doi:10.4230/LIPIcs.SoCG.2024.49.

[15] Tamal K. Dey, Fengtao Fan, and Yusu Wang. Computing topological persistence for simplicial maps. In *Proceedings of the 30th Annual Symposium on Computational Geometry (SoCG 2014)*, 2014.

[16] Paweł Dłotko, Tomasz Kaczynski, Marian Mrozek, and Thomas Wanner. Coreduction homology algorithm for regular CW-complexes. *Discrete & Computational Geometry*, 46(2):361–388, 2011. doi:10.1007/s00454-010-9303-y.

[17] Herbert Edelsbrunner and John L. Harer. *Computational Topology: An Introduction*. American Mathematical Society, Providence, RI, 2010. ISBN 978-0-8218-4925-5. doi:10.1090/mbk/069.

[18] Herbert Edelsbrunner, David Letscher, and Afra Zomorodian. Topological persistence and simplification. *Discrete & Computational Geometry*, 28(4):511–533, 2002. doi:10.1007/s00454-002-2885-2.

[19] David Eppstein. Dynamic generators of topologically embedded graphs. In *Proceedings of the 14th ACM-SIAM Symposium on Discrete Algorithms (SODA 2003)*, pages 599–608, 2003.

[20] Robin Forman. Morse theory for cell complexes. *Advances in Mathematics*, 134:90–145, 1998.

[21] Robin Forman. A user's guide to discrete Morse theory. *Séminaire Lotharingien de Combinatoire*, 48:Art. B48c, 2002.

[22] Barbara Giunti and Jānis Lazovskis. Pruning vineyards: Updating barcodes and representative cycles by removing simplices. *Foundations of Data Science*, 2026. doi:10.3934/fods.2026007. Early access; preprint arXiv:2312.03925.

[23] Shaun Harker, Konstantin Mischaikow, Marian Mrozek, and Vidit Nanda. Discrete Morse theoretic algorithms for computing homology of complexes and maps. *Foundations of Computational Mathematics*, 14:151–184, 2014. doi:10.1007/s10208-013-9145-0.

[24] Allen Hatcher. *Algebraic Topology*. Cambridge University Press, Cambridge, 2002.

[25] Jacob Holm, Kristian de Lichtenberg, and Mikkel Thorup. Poly-logarithmic deterministic fully-dynamic algorithms for connectivity, minimum spanning tree, 2-edge, and biconnectivity. *Journal of the ACM*, 48(4):723–760, 2001. doi:10.1145/502090.502095.

[26] Michael Jöllenbeck and Volkmar Welker. Minimal resolutions via algebraic discrete Morse theory. *Memoirs of the American Mathematical Society*, 197(923), 2009.

[27] Thomas Lewiner, Hélio Lopes, and Geovan Tavares. Optimal discrete Morse functions for 2-manifolds. *Computational Geometry*, 26(3):221–233, 2003.

[28] Yuan Luo and Bradley J. Nelson. Accelerating iterated persistent homology computations with warm starts. *Computational Geometry*, 2024. doi:10.1016/j.comgeo.2024.102089. Preprint arXiv:2108.05022, 2021.

[29] Clément Maria and Hannah Schreiber. Discrete Morse theory for computing zigzag persistence. *Discrete & Computational Geometry*, 71(2):708–737, 2024. Conference version in WADS 2019.

[30] Clément Maria, Jean-Daniel Boissonnat, Marc Glisse, and Mariette Yvinec. The Gudhi library: Simplicial complexes and persistent homology. In *Mathematical Software – ICMS 2014*, volume 8592 of *Lecture Notes in Computer Science*, pages 167–174. Springer, 2014.

[31] Nikola Milosavljević, Dmitriy Morozov, and Primož Škraba. Zigzag persistent homology in matrix multiplication time. In *Proceedings of the 27th Annual Symposium on Computational Geometry (SoCG 2011)*, pages 216–225, 2011. doi:10.1145/1998196.1998229.

[32] Konstantin Mischaikow and Vidit Nanda. Morse theory for filtrations and efficient computation of persistent homology. *Discrete & Computational Geometry*, 50(2):330–353, 2013. doi:10.1007/s00454-013-9529-6.

[33] Marian Mrozek and Bogdan Batko. Coreduction homology algorithm. *Discrete & Computational Geometry*, 41: 96–118, 2009.

[34] Marian Mrozek and Thomas Wanner. Coreduction homology algorithm for inclusions and persistent homology. *Computers & Mathematics with Applications*, 60(10):2812–2833, 2010.

[35] James R. Munkres. *Elements of Algebraic Topology*. Addison-Wesley, Menlo Park, CA, 1984.

[36] Nina Otter, Mason A. Porter, Ulrike Tillmann, Peter Grindrod, and Heather A. Harrington. A roadmap for the computation of persistent homology. *EPJ Data Science*, 6:17, 2017. doi:10.1140/epjds/s13688-017-0109-5.

[37] Udo Pachner. P.L. homeomorphic manifolds are equivalent by elementary shellings. *European Journal of Combinatorics*, 12(2):129–145, 1991. doi:10.1016/S0195-6698(13)80080-7.

[38] Emil Sköldberg. Morse theory from an algebraic viewpoint. *Transactions of the American Mathematical Society*, 358(1):115–129, 2006.

[39] Afra Zomorodian and Gunnar Carlsson. Computing persistent homology. *Discrete & Computational Geometry*, 33(2):249–274, 2005. doi:10.1007/s00454-004-1146-y.

A Full result tables

Latencies are per update in µs, the three runs pooled; $'$ marks gudhi on the prefix $T' = 200$, and an empty cell a method not run in that experiment. Medians, the E7 grid, per-seed ranges, noise and pairwise overlaps of seed ranges are in `experiments/results/` and `paper/generated/`.

Table 6: The build of K_0 in insertion order (drb-id) and in the Morse order (mmhm-norb): $\mathrm{nnz}(R) + \mathrm{nnz}(V)$ after INIT and column additions of the build, with the ratios Morse/insertion (E8).

			$\mathrm{nnz}(R) + \mathrm{nnz}(V)$ after INIT			additions of the INIT build		
exp.	instance	$\lvert K_0 \rvert$	insertion	Morse	ratio	insertion	Morse	ratio
E1	W2 $N = 7$, $s = 8$	10,739	39,978	39,783	0.995	192	0	0.00
E1	W2 $N = 10$, $s = 8$	28,757	107,651	107,454	0.998	192	0	0.00
E1	W2 $N = 15$, $s = 8$	92,787	349,191	348,999	0.999	192	0	0.00
E1	W2 $N = 22$, $s = 8$	286,589	1,082,490	1,082,298	1.000	192	0	0.00
E1	W2 $N = 33$, $s = 8$	955,059	3,617,055	3,616,863	1.000	192	0	0.00
E2	W2 $N = 22$, $s = 1$	285,805	1,079,466	1,079,442	1.000	24	0	0.00
E2	W2 $N = 22$, $s = 2$	285,917	1,079,898	1,079,850	1.000	48	0	0.00
E2	W2 $N = 22$, $s = 4$	286,141	1,080,762	1,080,666	1.000	96	0	0.00
E2	W2 $N = 22$, $s = 16$	287,485	1,085,946	1,085,562	1.000	384	0	0.00
E2	W2 $N = 22$, $s = 32$	289,277	1,092,858	1,092,090	0.999	768	0	0.00
E2	W2 $N = 22$, $s = 64$	292,861	1,106,686	1,105,146	0.999	1,536	0	0.00
E3	W3 $N = 14$	74,957	281,778	281,778	1.000	0	0	–
E3	W3 $N = 20$	215,321	812,580	812,580	1.000	0	0	–
E3	W3 $N = 27$	525,043	1,986,039	1,986,039	1.000	0	0	–
E4	W4 2D $n = 1$k, $D = 2$	16,095	64,379	64,071	0.995	24,751	26,973	1.09
E4	W4 3D $n = 1$k, $D = 3$	22,831	101,754	98,034	0.963	20,025	21,680	1.08
E4	W4 2D $n = 1$k, $D = 3$	28,435	127,170	124,894	0.982	31,045	32,392	1.04
E4	W4 2D $n = 4$k, $D = 2$	63,935	262,582	257,571	0.981	99,634	107,925	1.08
E4	W4 3D $n = 4$k, $D = 3$	85,205	381,901	368,940	0.966	68,575	78,678	1.15
E4	W4 2D $n = 4$k, $D = 3$	111,960	508,866	495,893	0.975	123,174	128,327	1.04
E4	W4 2D $n = 16$k, $D = 2$	254,196	1,062,015	1,041,464	0.981	396,613	427,675	1.08
E5	W1 sphere $L = 4$	15,362	51,227	48,636	0.949	5,134	5,119	1.00
E5	W1 torus 64×64	24,576	78,131	77,984	0.998	8,317	8,359	1.01
E5	W1 sphere $L = 5$	61,442	203,499	194,556	0.956	20,510	20,479	1.00
E5	W1 torus 128×128	98,304	311,923	311,628	0.999	33,021	33,107	1.00
E5	W1 sphere $L = 6$	245,762	810,635	778,236	0.960	81,982	81,919	1.00
E5	W1 torus 256×256	393,216	1,246,451	1,245,856	1.000	131,581	131,751	1.00
E5	W1 sphere $L = 7$	983,042	3,235,275	3,112,956	0.962	327,806	327,679	1.00
E5	W1 torus 500×500	1,500,000	4,752,487	4,751,324	1.000	500,997	501,331	1.00
E6	W5a $L = 1{,}000$	2,000	3,997	3,997	1.000	999	999	1.00
E6	W5b level 3	3,842	12,915	12,156	0.941	1,286	1,279	0.99
E6	W5a $L = 3{,}000$	6,000	11,997	11,997	1.000	2,999	2,999	1.00
E6	W5b level 4	15,362	51,227	48,636	0.949	5,134	5,119	1.00
E6	W5a $L = 10{,}000$	20,000	39,997	39,997	1.000	9,999	9,999	1.00
E6	W5a $L = 30{,}000$	60,000	119,997	119,997	1.000	29,999	29,999	1.00
E6	W5b level 5	61,442	203,499	194,556	0.956	20,510	20,479	1.00
E6	W5a $L = 100{,}000$	200,000	399,997	399,997	1.000	99,999	99,999	1.00
E6	W5b level 6	245,762	810,635	778,236	0.960	81,982	81,919	1.00
E6	W5a $L = 300{,}000$	600,000	1,199,997	1,199,997	1.000	299,999	299,999	1.00
E6	W5b level 7	983,042	3,235,275	3,112,956	0.962	327,806	327,679	1.00
E6	W5a $L = 1{,}000{,}000$	2,000,000	3,999,997	3,999,997	1.000	999,999	999,999	1.00
E6	W5b level 8	3,932,162	12,926,027	12,451,836	0.963	1,310,974	1,310,719	1.00

Table 7: Drift (Section 6.4): median latency of full-id over the first and the last 200 updates and their ratio; the same ratio for full-sorted, full-morse, drb-id and mmhm; and the speedup of mmhm over full-id and over full-sorted on each of the two windows (ratio of their amortised latencies over the same updates).

		full-id med.		last/first block median					mmhm × full-id		mmhm × full-sorted	
exp.	instance	first	last	f-id	f-sorted	f-morse	drb-id	mmhm	first	last	first	last
E1	W2 $N = 7$, $s = 8$	136	134	0.99	1.04	0.81	1.01	0.90	2.44	2.03	2.66	2.35
E1	W2 $N = 10$, $s = 8$	373	376	1.01	1.03	0.99	0.98	0.94	5.01	5.62	5.48	6.55
E1	W2 $N = 15$, $s = 8$	1,170	1,240	1.06	1.01	0.96	1.08	1.08	10.9	12.9	12.0	13.1
E1	W2 $N = 22$, $s = 8$	3,470	3,760	1.08	0.98	0.97	0.99	1.12	19.5	28.3	23.0	28.0
E1	W2 $N = 33$, $s = 8$	14,800	14,800	1.00	1.01	1.00	0.99	1.04	46.0	103	44.2	100
E2	W2 $N = 22$, $s = 1$	3,360	3,540	1.05		0.96	0.93	0.90	42.5	158		
E2	W2 $N = 22$, $s = 2$	3,480	3,090	0.89		0.92	0.80	0.92	33.7	95.4		
E2	W2 $N = 22$, $s = 4$	3,560	3,450	0.97		0.98	0.87	0.96	29.3	58.0		
E2	W2 $N = 22$, $s = 16$	3,460	3,670	1.06		0.94	0.93	0.96	11.1	16.5		
E2	W2 $N = 22$, $s = 32$	3,780	3,940	1.04		1.00	1.14	1.01	7.80	8.93		
E2	W2 $N = 22$, $s = 64$	4,150	5,020	1.21		0.96	1.12	1.26	5.58	5.45		
E3	W3 $N = 14$	1,680	2,400	1.44	1.44	1.33	2.00	1.22	1.76	1.95	1.33	1.67
E3	W3 $N = 20$	3,830	8,490	2.22	1.84	1.65	4.31	1.41	3.79	5.59	3.25	4.04
E3	W3 $N = 27$	8,040	23,000	2.86	1.72	1.50	4.18	1.20	7.79	15.0	7.45	8.44
E4	W4 2D $n = 1$k, $D = 2$	536	877	1.64	1.04	1.00	1.29	1.77	7.99	7.71	8.59	5.23
E4	W4 3D $n = 1$k, $D = 3$	751	1,000	1.33	1.01	1.15	1.34	2.12	4.96	3.51	5.60	2.82
E4	W4 2D $n = 1$k, $D = 3$	932	1,220	1.31	1.17	1.18	1.33	2.24	6.20	3.18	6.49	3.29
E4	W4 2D $n = 4$k, $D = 2$	2,140	3,190	1.49	1.13	1.08	1.34	2.08	22.4	18.0	29.5	17.7
E4	W4 3D $n = 4$k, $D = 3$	2,630	3,530	1.34	1.03	1.28	1.15	1.61	16.2	14.7	21.0	14.1
E4	W4 2D $n = 4$k, $D = 3$	3,570	5,200	1.46	1.23	1.27	1.13	1.52	15.1	14.5	17.8	14.2
E4	W4 2D $n = 16$k, $D = 2$	10,800	12,300	1.15	1.12	1.21	1.10	1.14	64.8	103	81.2	120
E5	W1 sphere $L = 4$	362	425	1.18	1.09	1.04	1.99	0.84	1.90	2.28	1.32	1.54
E5	W1 torus 64×64	410	752	1.83	1.00	1.13	2.16	0.90	1.44	2.07	1.50	1.16
E5	W1 sphere $L = 5$	1,500	1,730	1.15	0.99	1.02	2.03	0.90	2.13	1.88	2.16	1.80
E5	W1 torus 128×128	1,930	2,350	1.21	1.08	1.15	1.88	0.89	1.51	1.73	1.40	1.47
E5	W1 sphere $L = 6$	6,280	6,540	1.04	1.05	1.10	1.16	0.98	2.30	1.88	2.30	1.85
E5	W1 torus 256×256	7,750	8,530	1.10	0.99	1.21	1.21	1.00	1.20	1.19	1.36	1.21
E5	W1 sphere $L = 7$	27,700	29,700	1.07	1.01	1.04	1.17	0.84	1.95	1.79	2.06	1.78
E5	W1 torus 500×500	32,600	33,600	1.03	1.03	1.05	1.08	1.18	1.17	0.919	1.26	0.966
E6	W5a $L = 1{,}000$	22.4	22.8	1.02			0.62	0.96	1.96	2.11		
E6	W5b level 3	108	112	1.03			0.89	0.97	11.0	12.1		
E6	W5a $L = 3{,}000$	67.3	67.3	1.00			1.16	0.82	1.89	1.92		
E6	W5b level 4	617	590	0.96			0.91	0.91	13.5	14.4		
E6	W5a $L = 10{,}000$	246	235	0.95			1.01	0.77	1.75	1.58		
E6	W5a $L = 30{,}000$	788	770	0.98			0.82	1.50	1.97	1.63		
E6	W5b level 5	2,620	2,730	1.04			0.72	0.98	11.9	12.1		
E6	W5a $L = 100{,}000$	2,610	2,640	1.01			0.64	1.05	1.64	1.84		
E6	W5b level 6	10,400	11,000	1.05			0.64	0.94	8.60	9.47		
E6	W5a $L = 300{,}000$	8,050	8,100	1.01			0.75	1.08	1.33	1.34		
E6	W5b level 7	46,400	46,000	0.99			0.55	0.62	6.91	7.37		
E6	W5a $L = 1{,}000{,}000$	36,400	36,200	0.99			0.94	1.33	1.78	1.78		
E6	W5b level 8	188,000	191,000	1.01			0.80	0.60	5.75	6.49		

Table 8: Does a rebuild help? Instances on which the default policy ($\alpha = 1$, no period) rebuilt: rebuilds per run (rb: drb-id-rb, m: mmhm); ratios of amortised latency to the never-rebuilding twin (rb/id: drb-id-rb over drb-id, m/norb: mmhm over mmhm-norb); the same ratio of column additions over the 100 updates after each rebuild; and the ratios of additions and of time over all updates after the first rebuild. Above 1: the rebuilding method was costlier.

	rebuilds / run		amortized latency		adds, 100 upd. after		all updates after the first rebuild			
							rb/id		m/norb	
instance	rb	m	rb/id	m/norb	rb/id	m/norb	adds	time	adds	time
W3 $N = 14$	24	13	0.46	0.47	0.98	1.00	0.99	0.40	1.01	0.41
W3 $N = 20$	10	6.0	0.34	0.41	0.72	0.82	0.78	0.29	0.85	0.33
W3 $N = 27$	4.3	3.0	0.54	0.62	0.71	0.79	0.76	0.46	0.79	0.47
W4 3D $n = 1$k, $D = 3$	0.33	0.33	1.16	1.00	0.94	0.54	1.01	0.95	0.84	0.70
W1 sphere $L = 4$	18	21	0.56	0.55	0.68	1.19	0.71	0.50	1.13	0.48
W1 sphere $L = 5$	16	10	0.62	0.44	0.72	0.91	0.71	0.56	0.83	0.38
W1 sphere $L = 6$	11	5.7	0.93	1.12	1.31	1.36	1.08	0.90	0.95	1.10
W1 sphere $L = 7$	9.0	2.3	1.06	1.07	2.44	2.04	1.79	1.05	0.92	1.09
W1 torus 64×64	16	15	0.49	0.56	0.52	1.07	0.54	0.43	1.01	0.50
W1 torus 128×128	7.7	6.7	0.83	0.68	0.50	1.11	0.48	0.80	0.91	0.61
W1 torus 256×256	4.3	3.0	1.13	0.93	0.70	1.50	0.54	1.17	1.03	0.86
W1 torus 500×500	1.7	1.0	1.28	1.02	0.85	2.52	0.69	1.44	1.21	1.02

Table 9: Amortised per-update latency (µs) of every instance of E1–E6. The mmhm-norb entry for W2 with $N = 22$, $s = 8$ (in E1 and E2) includes the outlier run of Section 6.7. gudhi entries marked $'$ cover only the first T' updates and are not comparable with the others.

exp.	instance	$\lvert K_0\rvert$	full-id	full-sorted	full-morse	gudhi	drb-id	drb-id-rb	mmhm-norb	mmhm	euler
E1	W2 $N = 7$, $s = 8$	10,739	135	172	350	2,770	46.8	47.7	49.8	59.0	
E1	W2 $N = 10$, $s = 8$	28,757	387	413	944	8,190	56.5	59.7	65.3	71.6	
E1	W2 $N = 15$, $s = 8$	92,787	1,240	1,300	3,170	29,100	88.4	94.9	91.1	97.0	
E1	W2 $N = 22$, $s = 8$	286,589	3,700	3,960	11,100	97,600	122	118	171	128	
E1	W2 $N = 33$, $s = 8$	955,059	14,900	14,600	62,800	307,000′	165	163	163	165	
E2	W2 $N = 22$, $s = 1$	285,805	3,500		10,800		27.9	28.6	26.8	27.4	
E2	W2 $N = 22$, $s = 2$	285,917	3,470		10,800		39.3	38.8	39.7	41.2	
E2	W2 $N = 22$, $s = 4$	286,141	3,590		10,700		67.4	65.7	65.6	67.1	
E2	W2 $N = 22$, $s = 8$	286,589	3,700	3,960	11,100	97,600	122	118	171	128	
E2	W2 $N = 22$, $s = 16$	287,485	3,750		12,000		238	229	234	248	
E2	W2 $N = 22$, $s = 32$	289,277	4,100		12,600		430	440	450	461	
E2	W2 $N = 22$, $s = 64$	292,861	4,710		14,800		895	805	862	981	
E3	W3 $N = 14$	74,957	2,320	1,790	4,320	28,100	3,180	1,460	2,600	1,230	
E3	W3 $N = 20$	215,321	7,170	5,150	15,400	90,500	6,490	2,180	3,820	1,570	
E3	W3 $N = 27$	525,043	17,100	10,800	39,200	168,000′	4,600	2,500	2,920	1,810	
E4	W4 2D $n = 1$k, $D = 2$	16,095	822	655	1,100	6,280	115	116	107	122	
E4	W4 2D $n = 1$k, $D = 3$	28,435	1,190	1,120	1,810	13,000	296	301	275	289	
E4	W4 2D $n = 4$k, $D = 2$	63,935	2,830	2,960	4,850	34,700	140	146	113	126	
E4	W4 2D $n = 4$k, $D = 3$	111,960	4,610	4,990	8,980	73,200	376	370	277	299	
E4	W4 2D $n = 16$k, $D = 2$	254,196	12,000	14,400	25,100	196,000	169	168	110	127	
E4	W4 3D $n = 1$k, $D = 3$	22,831	1,060	924	1,590	11,000	244	284	241	241	
E4	W4 3D $n = 4$k, $D = 3$	85,205	3,160	3,440	6,290	51,300	236	245	192	201	
E5	W1 sphere $L = 4$	15,362	420	296	457	2,910	388	216	379	207	10.5
E5	W1 sphere $L = 5$	61,442	1,730	1,630	2,000	15,800	2,230	1,390	1,910	837	17.6
E5	W1 sphere $L = 6$	245,762	6,830	6,610	9,170	66,300	8,640	8,070	3,500	3,920	23.7
E5	W1 sphere $L = 7$	983,042	30,700	31,100	61,500	272,000′	37,100	39,200	15,500	16,600	29.5
E5	W1 torus 64×64	24,576	645	466	729	4,760	789	388	695	386	11.6
E5	W1 torus 128×128	98,304	2,260	2,060	3,570	24,400	2,950	2,450	2,420	1,650	18.5
E5	W1 torus 256×256	393,216	8,640	8,980	16,300	91,800′	10,300	11,600	7,680	7,170	26.2
E5	W1 torus 500×500	1,500,000	38,200	40,000	98,200	427,000′	45,100	57,600	33,400	34,000	30.1
E6	W5a $L = 1{,}000$	2,000	24.0			199	9.25	7.18	11.6	10.9	
E6	W5a $L = 3{,}000$	6,000	66.0			624	27.0	22.8	35.2	33.5	
E6	W5a $L = 10{,}000$	20,000	249			2,060	85.1	81.8	127	138	
E6	W5a $L = 30{,}000$	60,000	803			7,180	289	283	441	460	
E6	W5a $L = 100{,}000$	200,000	2,700			26,800	1,070	1,030	1,520	1,500	
E6	W5a $L = 300{,}000$	600,000	8,100			91,200′	3,500	3,590	5,910	5,950	
E6	W5a $L = 1{,}000{,}000$	2,000,000	36,300			376,000′	14,200	14,000	20,300	20,100	
E6	W5b level 3	3,842	118			762	9.72	11.1	10.6	9.67	0.106
E6	W5b level 4	15,362	604			2,860	40.9	42.0	40.6	41.6	0.127
E6	W5b level 5	61,442	2,520			12,600	229	232	205	214	0.142
E6	W5b level 6	245,762	10,200			61,500	951	951	1,120	1,100	0.173
E6	W5b level 7	983,042	43,800			270,000′	4,900	4,830	6,180	5,890	0.240
E6	W5b level 8	3,932,162	178,000			1,460,000′	21,500	21,700	29,400	28,700	0.267

Table 10: Counters between rebuilds (E8): median and p99 of the column additions and of the footprint ($\sum \lvert \mathrm{occ}(\sigma)\rvert$ over the deletions) per update, over the updates without a rebuild, and $\mathrm{nnz}(R) + \mathrm{nnz}(V)$ after INIT and at the end (means over the runs).

instance	method	additions / update		footprint / update		$\mathrm{nnz}(R) + \mathrm{nnz}(V)$	
		median	p99	median	p99	after INIT	end
W2 $N = 22$, $s = 8$	drb-id	156	1,420	118	228	1,080,000	1,090,000
	drb-id-rb	156	1,420	118	228	1,080,000	1,090,000
	mmhm-norb	166	1,050	88.5	259	1,080,000	1,090,000
	mmhm	166	1,050	88.5	259	1,080,000	1,090,000
W3 $N = 20$	drb-id	5,910	34,000	585	34,700	813,000	4,510,000
	drb-id-rb	4,990	19,000	634	13,400	813,000	1,670,000
	mmhm-norb	4,570	30,900	574	31,600	813,000	3,730,000
	mmhm	4,200	21,100	574	17,300	813,000	1,480,000
W4 2D $n = 16$k, $D = 2$	drb-id	276	2,130	204	1,910	1,060,000	1,260,000
	drb-id-rb	276	2,130	204	1,910	1,060,000	1,260,000
	mmhm-norb	204	1,130	60.0	801	1,040,000	1,300,000
	mmhm	204	1,130	60.0	801	1,040,000	1,300,000
W1 torus 256×256	drb-id	877	8,290	55.5	1,910	1,250,000	4,480,000
	drb-id-rb	370	7,620	56.0	618	1,250,000	1,740,000
	mmhm-norb	639	4,360	55.0	1,570	1,250,000	3,300,000
	mmhm	402	4,770	58.0	754	1,250,000	1,490,000

Table 11: 99th-percentile per-update latency (µs) of every instance of E1–E6. The mmhm-norb entry for W2 with $N = 22$, $s = 8$ includes the outlier run of Section 6.7.

exp.	instance	$\lvert K_0\rvert$	full-id	full-sorted	full-morse	gudhi	drb-id	drb-id-rb	mmhm-norb	mmhm	euler
E1	W2 $N = 7$, $s = 8$	10,739	235	289	610	4,550	113	130	112	129	
E1	W2 $N = 10$, $s = 8$	28,757	617	742	1,760	13,500	136	132	147	153	
E1	W2 $N = 15$, $s = 8$	92,787	1,810	1,900	5,040	46,400	205	230	177	188	
E1	W2 $N = 22$, $s = 8$	286,589	5,730	6,730	19,000	184,000	299	269	1,230	253	
E1	W2 $N = 33$, $s = 8$	955,059	21,600	21,900	97,900	386,000′	404	357	314	333	
E2	W2 $N = 22$, $s = 1$	285,805	5,280		18,900		74.1	77.7	72.0	68.0	
E2	W2 $N = 22$, $s = 2$	285,917	5,400		18,100		97.7	96.7	92.9	93.1	
E2	W2 $N = 22$, $s = 4$	286,141	5,140		17,700		168	161	133	137	
E2	W2 $N = 22$, $s = 8$	286,589	5,730	6,730	19,000	184,000	299	269	1,230	253	
E2	W2 $N = 22$, $s = 16$	287,485	6,030		20,600		537	535	443	455	
E2	W2 $N = 22$, $s = 32$	289,277	6,280		21,600		971	985	900	842	
E2	W2 $N = 22$, $s = 64$	292,861	7,240		28,600		2,250	1,670	1,640	1,930	
E3	W3 $N = 14$	74,957	3,590	3,550	7,000	46,900	15,900	10,400	13,500	5,110	
E3	W3 $N = 20$	215,321	12,500	9,710	29,000	143,000	47,300	9,720	27,200	6,500	
E3	W3 $N = 27$	525,043	30,800	20,100	69,800	232,000′	29,600	11,700	18,600	7,400	
E4	W4 2D $n = 1$k, $D = 2$	16,095	1,480	1,270	1,810	10,000	491	500	461	527	
E4	W4 2D $n = 1$k, $D = 3$	28,435	2,020	1,970	2,950	20,800	1,900	2,160	1,790	1,840	
E4	W4 2D $n = 4$k, $D = 2$	63,935	4,690	5,030	7,510	50,400	566	604	426	543	
E4	W4 2D $n = 4$k, $D = 3$	111,960	7,920	8,490	14,400	141,000	2,440	2,460	1,740	1,900	
E4	W4 2D $n = 16$k, $D = 2$	254,196	19,400	24,300	42,200	262,000	644	652	353	402	
E4	W4 3D $n = 1$k, $D = 3$	22,831	1,750	1,700	2,320	18,800	2,000	2,160	1,920	1,820	
E4	W4 3D $n = 4$k, $D = 3$	85,205	4,920	6,190	10,400	76,400	1,490	1,460	1,020	1,020	
E5	W1 sphere $L = 4$	15,362	778	703	1,050	4,310	2,550	2,030	2,390	1,780	36.6
E5	W1 sphere $L = 5$	61,442	3,190	4,110	4,720	26,600	21,900	15,400	21,200	8,300	55.2
E5	W1 sphere $L = 6$	245,762	12,700	12,800	22,000	101,000	80,500	84,700	31,300	38,600	64.1
E5	W1 sphere $L = 7$	983,042	52,200	53,500	114,000	333,000′	305,000	355,000	128,000	151,000	73.9
E5	W1 torus 64×64	24,576	1,210	1,160	1,850	7,840	6,580	3,620	5,800	3,860	42.7
E5	W1 torus 128×128	98,304	4,830	4,880	8,600	38,500	26,800	20,900	25,800	19,400	59.4
E5	W1 torus 256×256	393,216	19,800	19,800	38,300	127,000′	78,800	89,400	92,000	88,400	68.2
E5	W1 torus 500×500	1,500,000	79,000	78,800	183,000	546,000′	313,000	420,000	401,000	397,000	74.5
E6	W5a $L = 1{,}000$	2,000	54.2			344	173	143	73.3	70.3	
E6	W5a $L = 3{,}000$	6,000	96.8			903	486	376	230	238	
E6	W5a $L = 10{,}000$	20,000	379			3,080	1,360	1,340	809	952	
E6	W5a $L = 30{,}000$	60,000	1,210			10,600	4,270	4,260	2,750	3,170	
E6	W5a $L = 100{,}000$	200,000	3,710			38,200	14,500	14,300	9,170	9,110	
E6	W5a $L = 300{,}000$	600,000	11,800			121,000′	44,600	45,500	29,200	29,500	
E6	W5a $L = 1{,}000{,}000$	2,000,000	48,100			457,000′	154,000	151,000	103,000	101,000	
E6	W5b level 3	3,842	216			1,190	120	139	172	188	0.202
E6	W5b level 4	15,362	983			4,560	463	469	656	665	0.747
E6	W5b level 5	61,442	3,760			18,800	2,960	2,980	2,930	3,020	1.40
E6	W5b level 6	245,762	15,700			87,800	9,910	11,800	13,800	13,700	2.24
E6	W5b level 7	983,042	64,900			338,000′	51,000	50,700	63,300	61,500	3.54
E6	W5b level 8	3,932,162	240,000			1,650,000′	203,000	206,000	276,000	270,000	4.07

Table 12: W4: ranges of the Betti numbers over all states of the three runs, and β_D/n at INIT. β_D is dominated by unfilled boundaries of $(D+2)$-cliques (Section 6.3).

instance	$\lvert K_0\rvert$	β_0	β_1	β_2	β_3	β_D/n at INIT
W4 2D $n = 1$k, $D = 2$	16,095	1–2	61–86	5,696–6,612	0	6.17
W4 2D $n = 4$k, $D = 2$	63,935	1–3	308–353	23,714–25,167	0	6.07
W4 2D $n = 16$k, $D = 2$	254,196	2–5	1,343–1,410	95,351–96,175	0	5.97
W4 2D $n = 1$k, $D = 3$	28,435	1–2	61–86	0	4,960–7,441	6.17
W4 2D $n = 4$k, $D = 3$	111,960	1–3	308–353	0–1	22,252–25,967	5.94
W4 3D $n = 1$k, $D = 3$	22,831	1–4	133–165	0–10	3,214–5,259	3.51
W4 3D $n = 4$k, $D = 3$	85,205	1–7	654–745	8–27	10,091–12,104	2.83

Table 13: Build order (E8): one instance per family built in insertion order (drb-id), in the Morse order (mmhm-norb) and in a random order (drb-rand): $\mathrm{nnz}(R) + \mathrm{nnz}(V)$ after INIT, column additions of the build, and amortised latency (three runs pooled).

		$\mathrm{nnz}(R) + \mathrm{nnz}(V)$ after INIT			additions of the INIT build			amortized latency (µs)		
instance	$\lvert K_0\rvert$	insertion	Morse	random	insertion	Morse	random	insertion	Morse	random
W1 torus 128×128	98,304	311,923	311,628	624,407	33,021	33,107	48,963	2,950	2,420	8,740
W2 $N = 15$, $s = 8$	92,787	349,191	348,999	794,651	192	0	72,361	88.4	91.1	348
W3 $N = 14$	74,957	281,778	281,778	645,340	0	0	59,696	3,180	2,600	4,760
W4 2D $n = 4$k, $D = 2$	63,935	262,582	257,571	482,480	99,634	107,925	212,567	140	113	167